

Disciplina:

# **MACHINE LEARNING**

Professor: Rafael Barroso

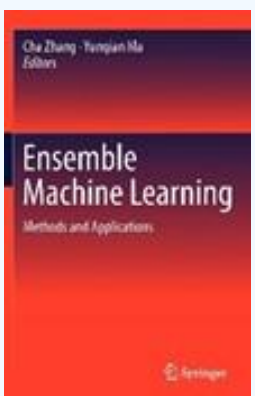


# TÓPICO 1

## FUNDAMENTOS DE ENSEMBLE: POR QUE COMBINAR MODELOS?

# TIPOS DE COMBINAÇÃO DE MODELOS

- Técnica que combina vários modelos de aprendizado de máquina com o objetivo de melhorar o desempenho geral.
- A redução da variabilidade pode ser pensada como a redução de ruído (alta variância) usando um filtro de média móvel.
- Combinar as saídas do classificador não leva necessariamente a um desempenho de classificação que seja garantido ser melhor do que o melhor classificador no conjunto. Em vez disso, reduz nossa probabilidade de escolher um classificador com um desempenho ruim.



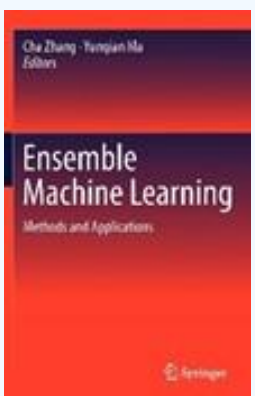
# TIPOS DE COMBINAÇÃO DE MODELOS

- **VANTAGENS:**

- Melhor desempenho
- Redução de variância
- Resistência a outliers

- **DESVANTAGENS:**

- Requerem conjuntos de dados maiores para treinamento
- Computacionalmente mais caros
- Possuem maior complexidade na interpretação



# TIPOS DE COMBINAÇÃO DE MODELOS

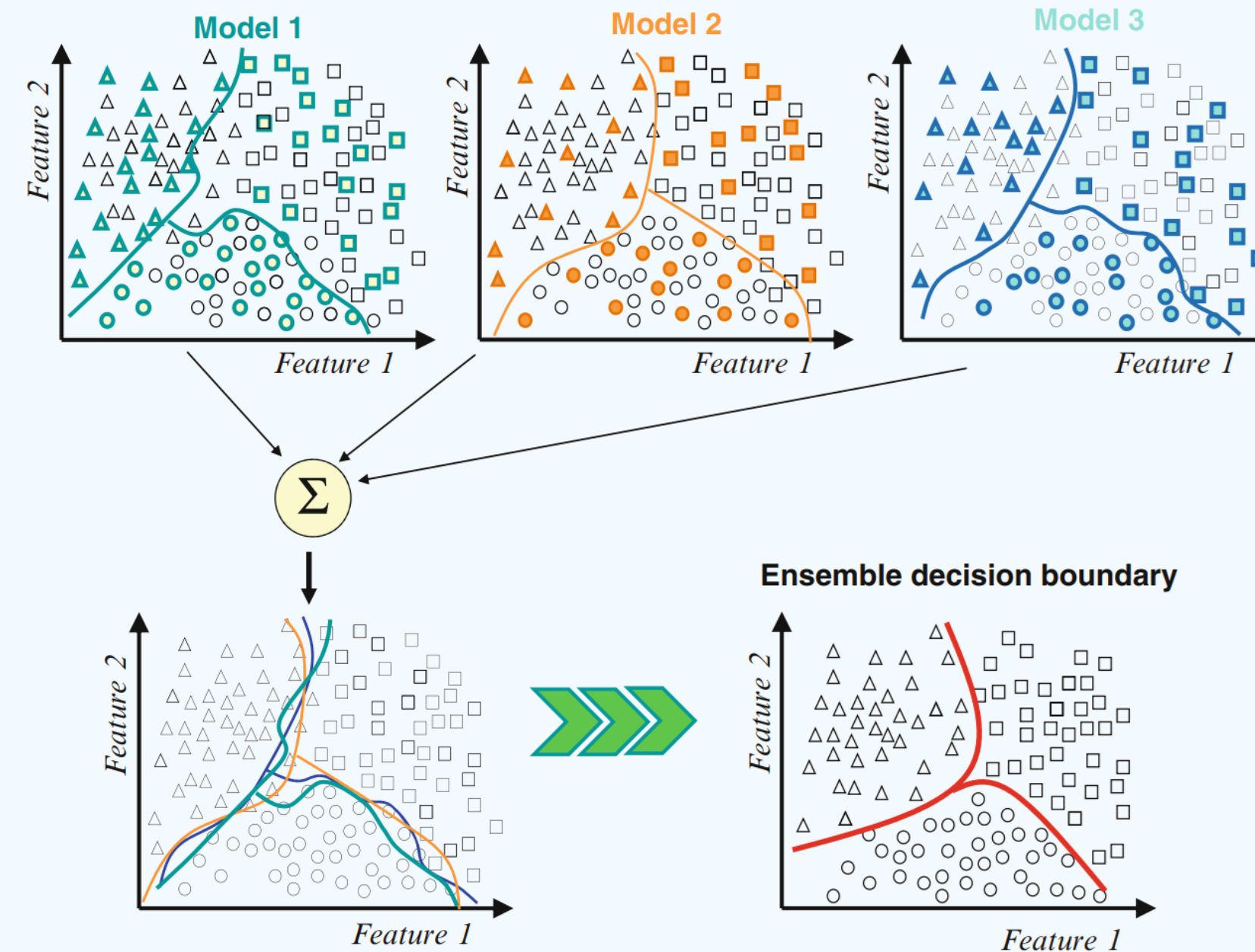


Fig. 1.1 Variability reduction using ensemble systems

# TIPOS DE COMBINAÇÃO DE MODELOS

## ■ PREMISSAS E ESTRATÉGIAS:

### DIVERSIDADE

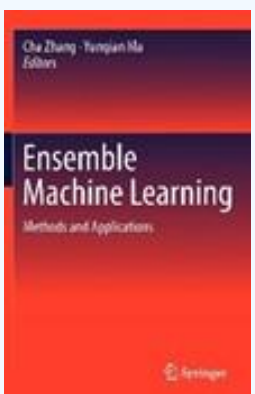
- A diversidade nos erros dos membros do conjunto é de suma importância;
- Idealmente, as saídas do classificador devem ser independentes;
- Usar diferentes subconjuntos dos dados de treinamento é a abordagem mais comum;
- Por outro lado, também se pode usar diferentes subconjuntos de atributos, gerando modelos treinados em subespaços diferentes.

### REDUÇÃO DE VARIÂNCIA

- A variância é uma medida da incerteza no modelo. Um modelo com alta variância é mais propenso a cometer erros;
- Modelos individuais são propensos a cometer erros diferentes. Ao combinar suas saídas, os erros são, em média, compensados.

### COMBINAÇÃO

- **Bagging:** cada membro é treinado em um subconjunto aleatório dos dados de treino;
- **Stacking:** os membros são treinados em conjuntos de dados diferentes e, em seguida, suas saídas são combinadas usando um modelo de aprendizado de máquina final;
- **Boosting:** cada membro é treinado para corrigir os erros dos modelos anteriores.



# TIPOS DE COMBINAÇÃO DE MODELOS

## BAGGING

### ■ PASSO 1: SELEÇÃO DOS DADOS DE TREINAMENTO

Divisão dos dados de treinamento em  $m$  subconjuntos aleatórios de tamanho  $n/m$ . Isso é feito usando um algoritmo de amostragem aleatória com reposição. A reposição significa que um ponto de dados pode ser selecionado mais de uma vez para um subconjunto.

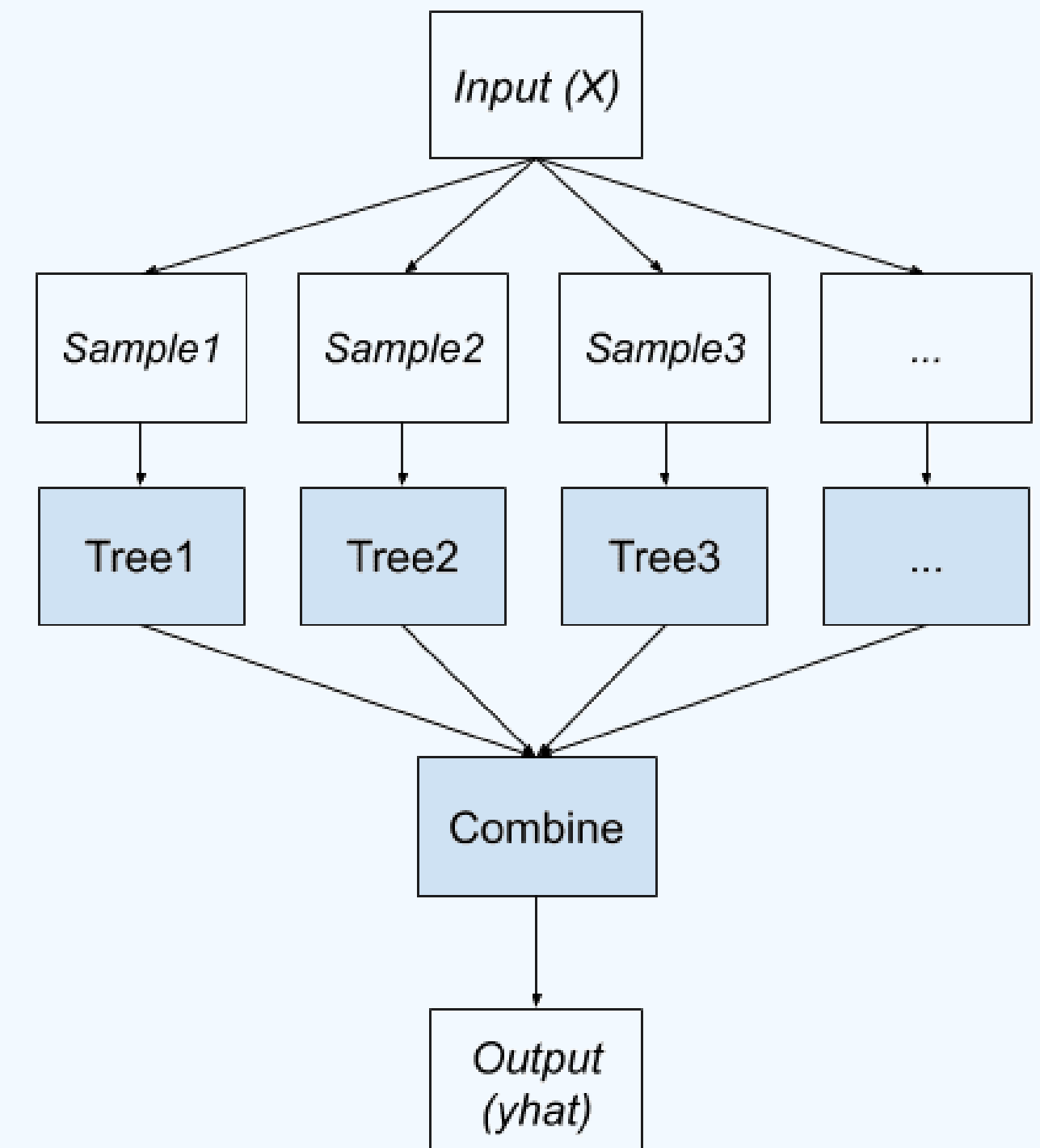
### ■ PASSO 2: TREINAMENTO DOS MEMBROS

Cada modelo individual é treinado em um subconjunto aleatório dos dados de treinamento. O mesmo algoritmo é usado para treinar todos os modelos individuais.

### ■ PASSO 3: COMBINAÇÃO DAS SAÍDAS DOS MEMBROS

As saídas dos modelos individuais são combinados, sendo a estratégia de combinação mais comum é a média.

### Bagging Ensemble



# TIPOS DE COMBINAÇÃO DE MODELOS

## STACKING

### ■ PASSO 1: SELEÇÃO DOS DADOS DE TREINAMENTO

O primeiro passo é dividir os dados de treinamento em dois conjuntos: um para os modelos individuais (maior) e um para o modelo final (menor).

### ■ PASSO 2: TREINAMENTO DOS MEMBROS

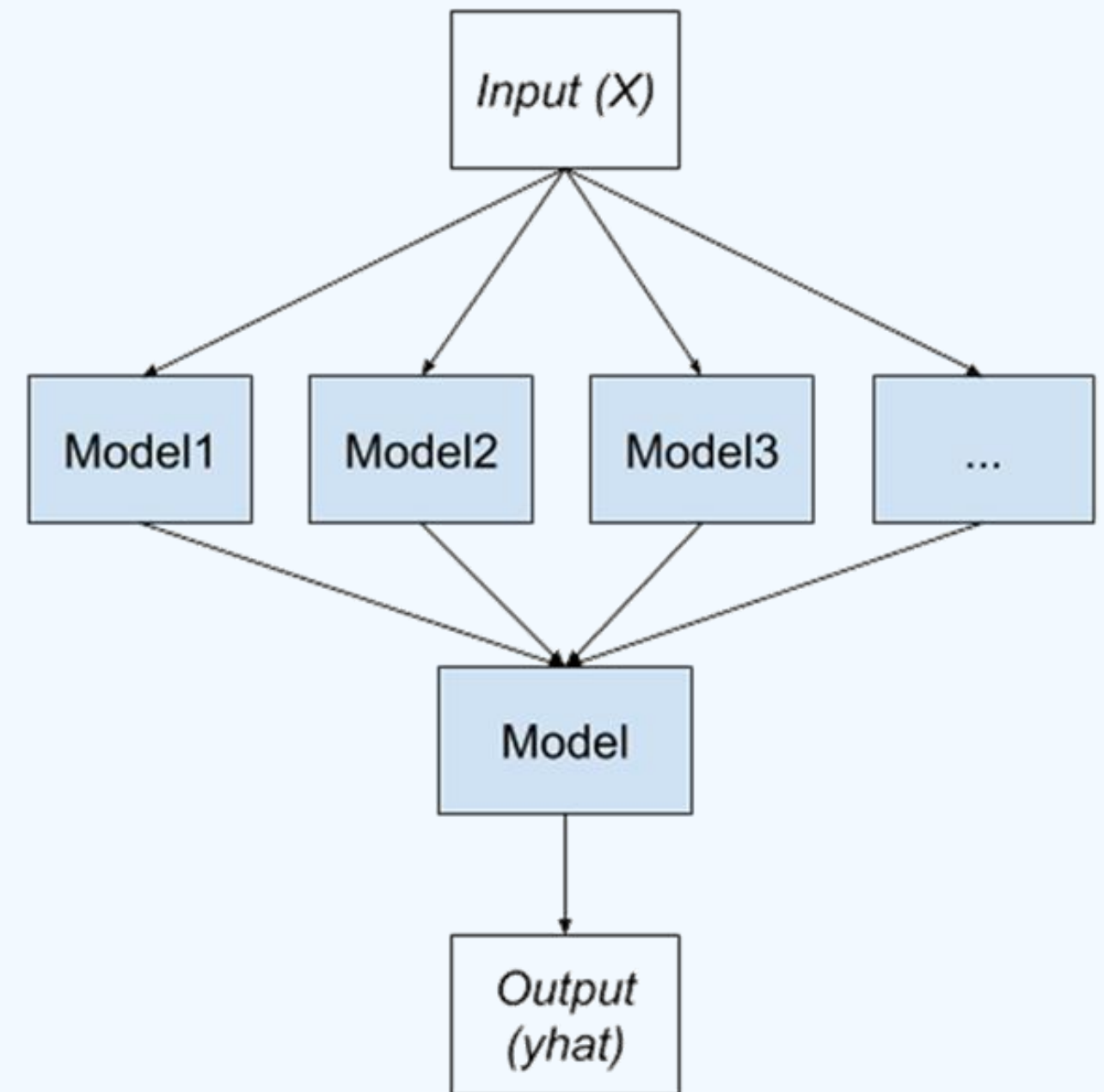
Cada modelo individual é treinado com o conjunto de dados de treinamento.

Diferentes algoritmos/modelos são usados para treinar cada o modelo individual.

### ■ PASSO 3: GERAÇÃO DOS ATRIBUTOS

Os atributos dos modelos individuais são gerados usando as saídas dos modelos individuais no conjunto de treinamento para o modelo final.

Stacking Ensemble



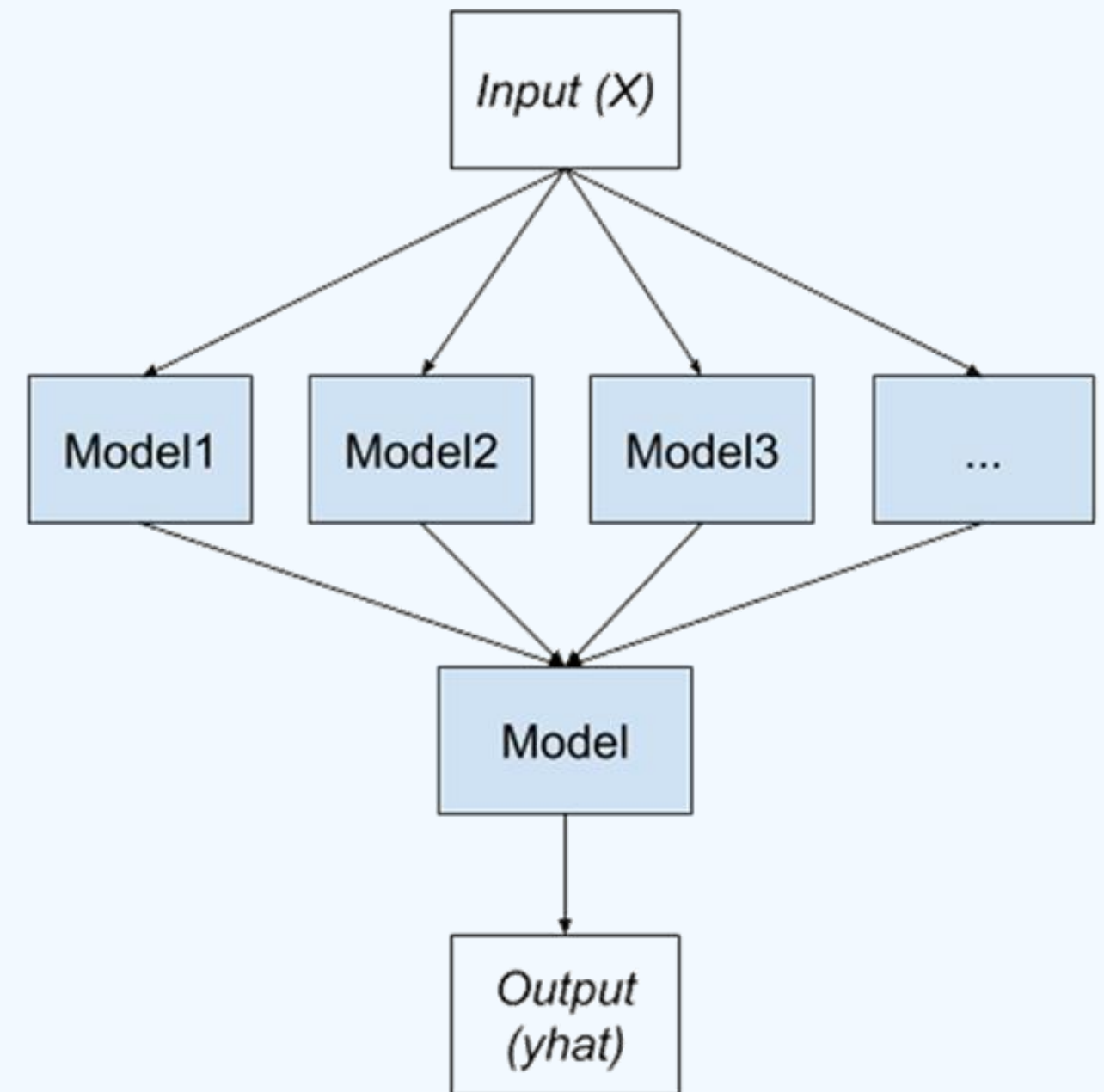
# TIPOS DE COMBINAÇÃO DE MODELOS

## STACKING

### ▪ PASSO 4: TREINAMENTO DO MODELO FINAL

O modelo final é treinado no conjunto de treinamento para o modelo final usando os atributos dos modelos individuais como entrada. O modelo final pode ser qualquer algoritmo de aprendizado de máquina, mas tente a ser uma regressão logística para problemas de classificação binária ou uma regressão linear para problemas de regressão.

Stacking Ensemble



# TIPOS DE COMBINAÇÃO DE MODELOS

## BOOSTING

### ■ PASSO 1: TREINAMENTO DO PRIMEIRO MODELO

É treinado no conjunto de treinamento original. O mesmo algoritmo de aprendizado de máquina é usado para treinar todos os modelos individuais.

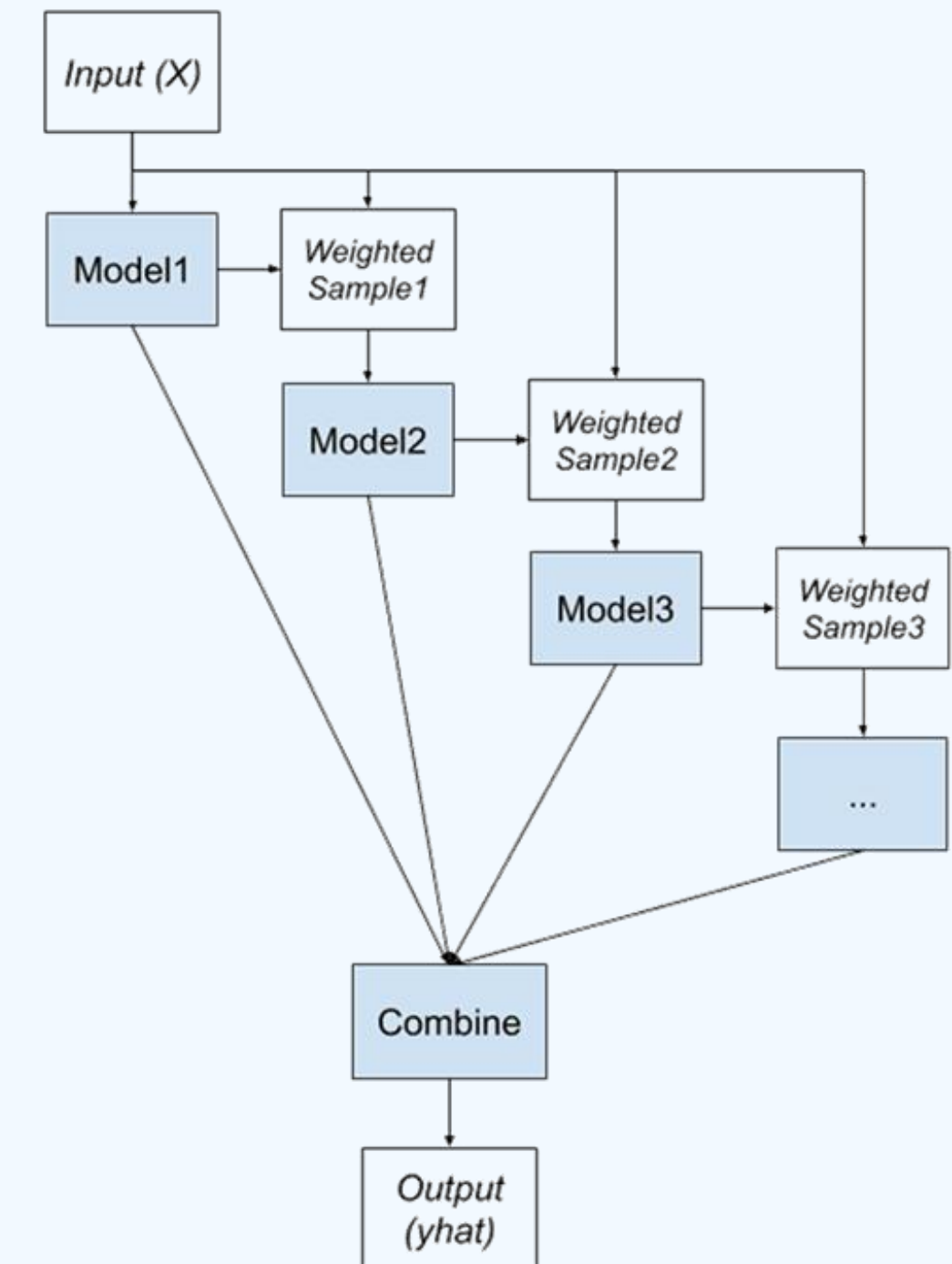
### ■ PASSO 2: CÁLCULO DOS PESOS

Pesos são calculados para cada ponto de dados, com pesos maiores sendo atribuídos aos pontos de dados que foram classificados incorretamente pelo primeiro modelo. Os pesos podem ser calculados usando uma variedade de técnicas.

### ■ PASSO 3: TREINAMENTO DO SEGUNDO MODELO

O segundo modelo é treinado em um conjunto de treinamento ponderado, com pontos de dados com pesos maiores recebendo mais atenção.

Boosting Ensemble



# TIPOS DE COMBINAÇÃO DE MODELOS

## BOOSTING

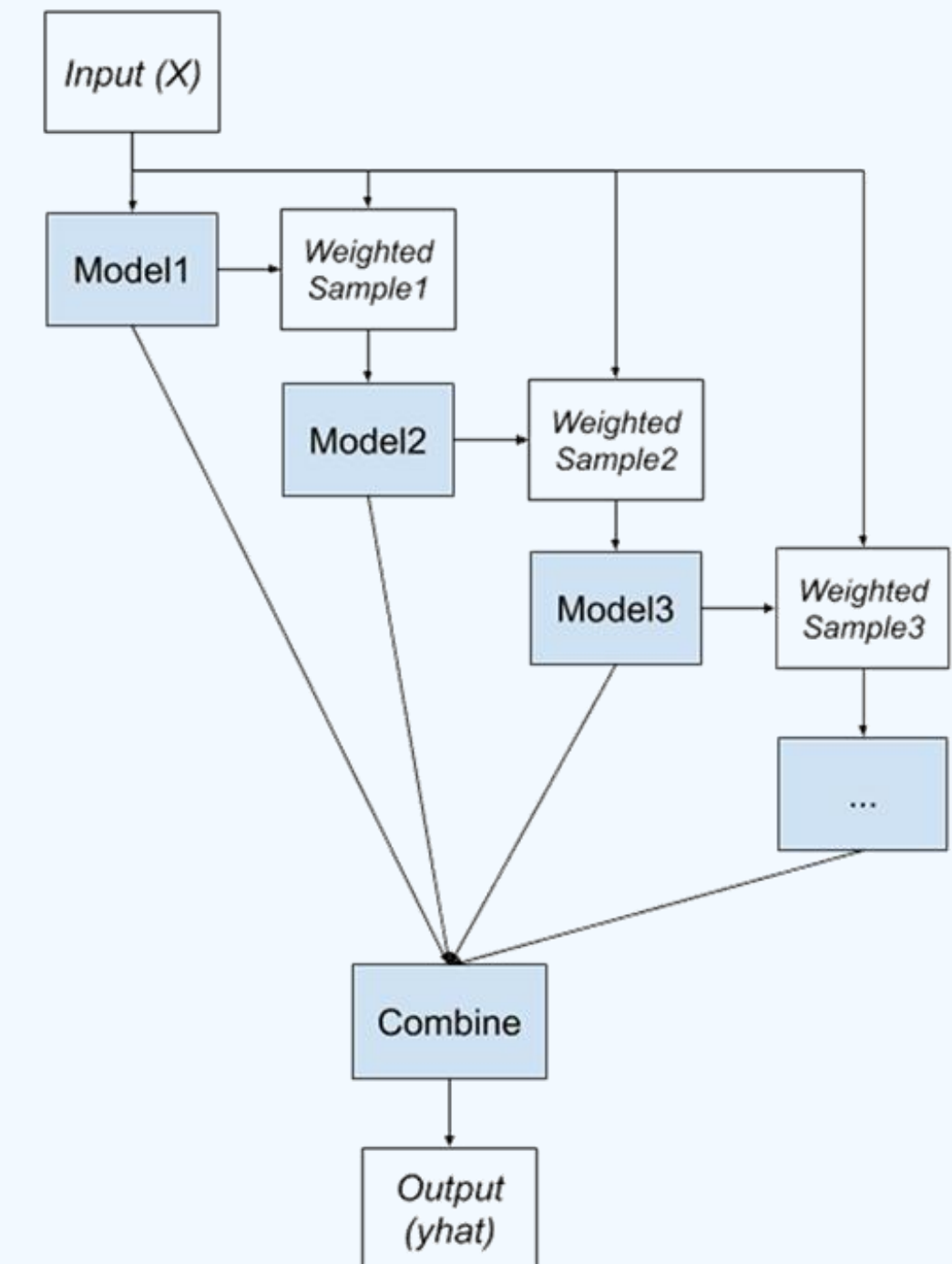
### ■ PASSO 4: REPETIÇÃO DOS PASSOS 2 E 3

Os passos 2 e 3 são repetidos para cada modelo individual, com o segundo modelo sendo treinado para corrigir os erros do primeiro modelo, o terceiro modelo sendo treinado para corrigir os erros do segundo modelo, e assim por diante.

### ■ PASSO 5: COMBINAÇÃO DAS SAÍDAS DOS MODELOS

As saídas dos modelos individuais são combinadas usando uma estratégia de combinação, como a média ou a votação.

Boosting Ensemble



# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

**BAGGING**

**RANDOM FOREST**

**BOOSTING:  
ADABOOST, GBM,  
XGBOOST,  
LIGHTGBM E  
CATBOOST**

**STACKING**

# TÓPICO 2

## **BAGGING: RANDOM FOREST E AMOSTRAGEM BOOTSTRAP**

# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

### BAGGING

- Visa reduzir a **variância** de modelos com alta complexidade (como árvores de decisão profundas), diminuindo o risco de *overfitting*.
- Ele treina múltiplos modelos independentes em diferentes subconjuntos de dados e agrega suas predições.
- **Funcionamento:**
  - **Bootstrap (Amostragem com Reposição):** A partir do conjunto de dados original de tamanho N, crie B novos conjuntos de dados (amostras *bootstrap*), cada um com tamanho N. Cada amostra é criada selecionando-se aleatoriamente observações do conjunto original com reposição. Isso significa que algumas observações podem aparecer várias vezes, enquanto outras podem não aparecer (são chamadas de ***out-of-bag***).
  - **Treinamento Paralelo:** Treine B modelos (geralmente do mesmo tipo) de forma independente e paralela, um para cada amostra *bootstrap*.

# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

### BAGGING

- **Funcionamento:**

- **Agregação (*Aggregating*):** Combine as previsões dos B modelos. Para Regressão a previsão final é a média das previsões de todos os modelos; para Classificação a previsão final é a classe mais votada (moda) entre os modelos.

- **Pontos Positivos:**

- Reduz efetivamente o *overfitting*.
- Pode ser totalmente paralelizado, acelerando o treinamento.
- Simples de implementar.

- **Pontos Negativos:**

- Não reduz o viés. Se o modelo base for fraco (alto viés), o Bagging não irá melhorá-lo.
- Perde-se a interpretabilidade de um modelo único.
- Aumenta o custo computacional em relação a um único modelo.

# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

## BAGGING

```

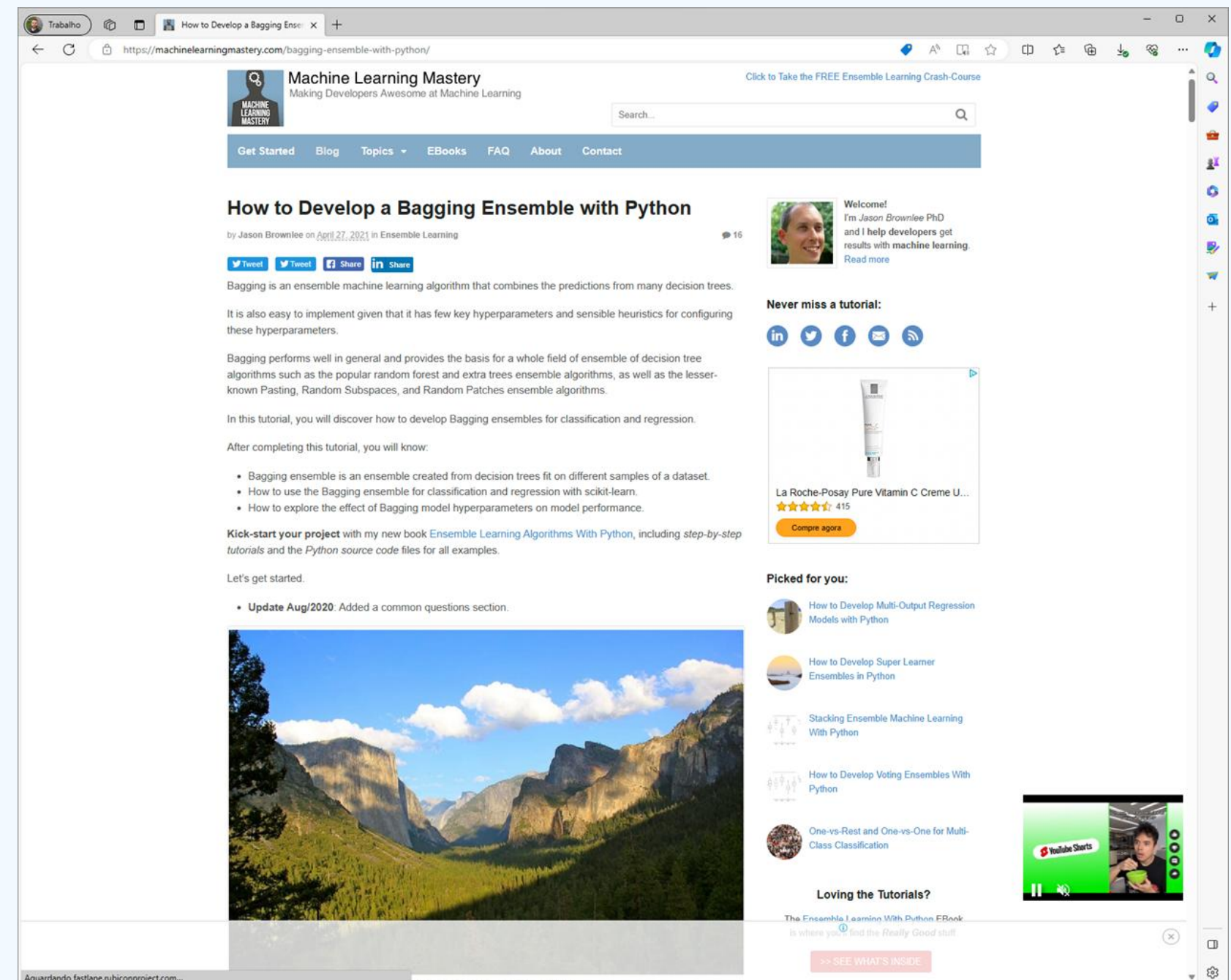
1 # evaluate bagging algorithm for classification
2 from numpy import mean
3 from numpy import std
4 from sklearn.datasets import make_classification
5 from sklearn.model_selection import cross_val_score
6 from sklearn.model_selection import RepeatedStratifiedKFold
7 from sklearn.ensemble import BaggingClassifier
8 # define dataset
9 X, y = make_classification(n_samples=1000, n_features=20, n_informative=15, n_redundant=5, random_state=5)
10 # define the model
11 model = BaggingClassifier()
12 # evaluate the model
13 cv = RepeatedStratifiedKFold(n_splits=10, n_repeats=3, random_state=1)
14 n_scores = cross_val_score(model, X, y, scoring='accuracy', cv=cv, n_jobs=-1, error_score='raise')
15 # report performance
16 print('Accuracy: %.3f (%.3f)' % (mean(n_scores), std(n_scores)))

```

```

1 # make predictions using bagging for classification
2 from sklearn.datasets import make_classification
3 from sklearn.ensemble import BaggingClassifier
4 # define dataset
5 X, y = make_classification(n_samples=1000, n_features=20, n_informative=15, n_redundant=5, random_state=5)
6 # define the model
7 model = BaggingClassifier()
8 # fit the model on the whole dataset
9 model.fit(X, y)
10 # make a single prediction
11 row = [[-4.7705504, -1.88685058, -0.96057964, 2.53850317, -6.5843005, 3.45711663, -7.46225013, 2.01338213, -0.45086384, -1.89314931, -2.90675203, -0.21214568, -0.9623956, 3.93862591, 0.06276375, 0.33964269, 4.0835676, 1.31423977, -2.17983117, 3.1047287]]
12 yhat = model.predict(row)
13 print('Predicted Class: %d' % yhat[0])

```



The screenshot shows the Machine Learning Mastery website. The main article is titled "How to Develop a Bagging Ensemble with Python" by Jason Brownlee. The article discusses Bagging as an ensemble machine learning algorithm that combines predictions from many decision trees. It mentions that Bagging performs well in general and provides the basis for a whole field of ensemble of decision tree algorithms such as the popular random forest and extra trees ensemble algorithms, as well as the lesser-known Pasting, Random Subspaces, and Random Patches ensemble algorithms. The article also mentions that in this tutorial, you will discover how to develop Bagging ensembles for classification and regression. After completing this tutorial, you will know:

- Bagging ensemble is an ensemble created from decision trees fit on different samples of a dataset.
- How to use the Bagging ensemble for classification and regression with scikit-learn.
- How to explore the effect of Bagging model hyperparameters on model performance.

The article also mentions a kick-start your project with my new book "Ensemble Learning Algorithms With Python", including step-by-step tutorials and the Python source code files for all examples. Let's get started.

Update Aug/2020: Added a common questions section.

The article includes a large image of a mountain landscape. On the right side, there is a sidebar with a "Never miss a tutorial:" section featuring social media links and a "Picked for you:" section with links to other tutorials. At the bottom right, there is a "Loving the Tutorials?" section with a link to "The Ensemble Learning With Python eBook".

# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

## BAGGING

Trabalho

sklearn.ensemble.BaggingClassi

+

https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.BaggingClassifier.html#sklearn.ensemble.BaggingClassifier

scikit-learn

Install User Guide API Examples Community More

Prev Up Next

scikit-learn 1.3.2  
Other versions

Please cite us if you use the software.

sklearn.ensemble.BaggingClassifi  
er  
BaggingClassifier

sklearn.ensemble.BaggingClassifier

```
class sklearn.ensemble.BaggingClassifier(estimator=None, n_estimators=10, *, max_samples=1.0, max_features=1.0, bootstrap=True, bootstrap_features=False, oob_score=False, warm_start=False, n_jobs=None, random_state=None, verbose=0, base_estimator='deprecated') [source]
```

A Bagging classifier.

A Bagging classifier is an ensemble meta-estimator that fits base classifiers each on random subsets of the original dataset and then aggregate their individual predictions (either by voting or by averaging) to form a final prediction. Such a meta-estimator can typically be used as a way to reduce the variance of a black-box estimator (e.g., a decision tree), by introducing randomization into its construction procedure and then making an ensemble out of it.

This algorithm encompasses several works from the literature. When random subsets of the dataset are drawn as random subsets of the samples, then this algorithm is known as Pasting [1]. If samples are drawn with replacement, then the method is known as Bagging [2]. When random subsets of the dataset are drawn as random subsets of the features, then the method is known as Random Subspaces [3]. Finally, when base estimators are built on subsets of both samples and features, then the method is known as Random Patches [4].

Read more in the User Guide.

New in version 0.15.

Parameters:

**estimator : object, default=None**  
The base estimator to fit on random subsets of the dataset. If None, then the base estimator is a `DecisionTreeClassifier`.

New in version 1.2: `base_estimator` was renamed to `estimator`.

**n\_estimators : int, default=10**  
The number of base estimators in the ensemble.

**max\_samples : int or float, default=1.0**  
The number of samples to draw from X to train each base estimator (with replacement by default, see `bootstrap` for more details).

- If int, then draw `max_samples` samples.
- If float, then draw `max_samples * X.shape[0]` samples.

**max\_features : int or float, default=1.0**  
The number of features to draw from X to train each base estimator ( without replacement by default, see `bootstrap_features` for more details).

- If int, then draw `max_features` features.
- If float, then draw `max(1, int(max_features * n_features_in_))` features.

**bootstrap : bool, default=True**  
Whether samples are drawn with replacement. If False, sampling without replacement is performed.

**bootstrap\_features : bool, default=False**  
Whether features are drawn with replacement.

**oob\_score : bool, default=False**  
Whether to use out-of-bag samples to estimate the generalization error. Only available if `bootstrap=True`.

Trabalho

sklearn.ensemble.BaggingRegre

+

https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.BaggingRegressor.html#sklearn.ensemble.BaggingRegressor

scikit-learn

Install User Guide API Examples Community More

Prev Up Next

scikit-learn 1.3.2  
Other versions

Please cite us if you use the software.

sklearn.ensemble.BaggingRegress  
or  
BaggingRegressor  
Examples using  
sklearn.ensemble.BaggingRegressor

sklearn.ensemble.BaggingRegressor

```
class sklearn.ensemble.BaggingRegressor(estimator=None, n_estimators=10, *, max_samples=1.0, max_features=1.0, bootstrap=True, bootstrap_features=False, oob_score=False, warm_start=False, n_jobs=None, random_state=None, verbose=0, base_estimator='deprecated') [source]
```

A Bagging regressor.

A Bagging regressor is an ensemble meta-estimator that fits base regressors each on random subsets of the original dataset and then aggregate their individual predictions (either by voting or by averaging) to form a final prediction. Such a meta-estimator can typically be used as a way to reduce the variance of a black-box estimator (e.g., a decision tree), by introducing randomization into its construction procedure and then making an ensemble out of it.

This algorithm encompasses several works from the literature. When random subsets of the dataset are drawn as random subsets of the samples, then this algorithm is known as Pasting [1]. If samples are drawn with replacement, then the method is known as Bagging [2]. When random subsets of the dataset are drawn as random subsets of the features, then the method is known as Random Subspaces [3]. Finally, when base estimators are built on subsets of both samples and features, then the method is known as Random Patches [4].

Read more in the User Guide.

New in version 0.15.

Parameters:

**estimator : object, default=None**  
The base estimator to fit on random subsets of the dataset. If None, then the base estimator is a `DecisionTreeRegressor`.

New in version 1.2: `base_estimator` was renamed to `estimator`.

**n\_estimators : int, default=10**  
The number of base estimators in the ensemble.

**max\_samples : int or float, default=1.0**  
The number of samples to draw from X to train each base estimator (with replacement by default, see `bootstrap` for more details).

- If int, then draw `max_samples` samples.
- If float, then draw `max_samples * X.shape[0]` samples.

**max\_features : int or float, default=1.0**  
The number of features to draw from X to train each base estimator ( without replacement by default, see `bootstrap_features` for more details).

- If int, then draw `max_features` features.
- If float, then draw `max(1, int(max_features * n_features_in_))` features.

**bootstrap : bool, default=True**  
Whether samples are drawn with replacement. If False, sampling without replacement is performed.

**bootstrap\_features : bool, default=False**  
Whether features are drawn with replacement.

**oob\_score : bool, default=False**  
Whether to use out-of-bag samples to estimate the generalization error. Only available if `bootstrap=True`.

# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

### RANDOM FOREST

- É uma evolução do Bagging que usa árvores de decisão como modelos base. Adiciona uma camada extra de aleatoriedade para "decorrelacionar" ainda mais as árvores, resultando em uma redução de variância ainda maior.
- **Funcionamento:**
  - **Bootstrap:** Idêntico ao Bagging, cria-se B amostras *bootstrap*.
  - **Treinamento com Aleatoriedade de Features:** Para cada uma das B árvores:

Ao construir a árvore, em cada nó (divisão), em vez de procurar a melhor feature entre todas as features disponíveis, o algoritmo seleciona um subconjunto aleatório de m features (onde m é um hiperparâmetro, tipicamente  $m \cong \sqrt{p}$  para classificação, sendo p o total de features). Em seguida, encontra a melhor feature e o melhor ponto de corte para a divisão apenas dentro desse subconjunto.
  - **Agregação:** Idêntico ao Bagging (média para regressão, votação para classificação).

# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

### RANDOM FOREST

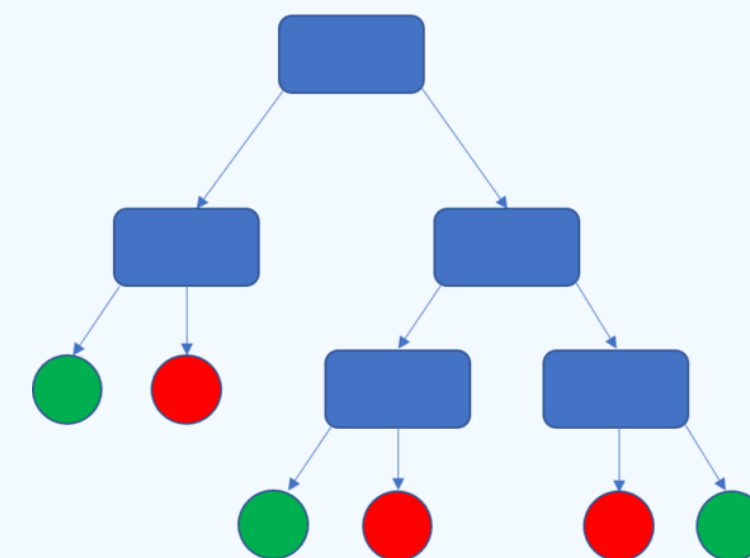
- No Bagging com árvores, se uma feature é muito dominante, a maioria das árvores a escolherá como a primeira divisão, tornando-as estruturalmente semelhantes e correlacionadas.
- Ao forçar cada divisão a considerar apenas um subconjunto de features, o Random Forest garante que a feature dominante não seja sempre uma opção. Outras features terão a chance de serem escolhidas, criando árvores muito mais diversas e, portanto, diminuindo drasticamente a correlação.
- **Pontos Positivos:**
  - Performance de ponta com pouco ajuste de hiperparâmetros.
  - Extremamente robusto a *overfitting*.
  - Fornece métricas de importância das features (*feature importance*).
  - Lida bem com dados faltantes.

# PRINCIPAIS ALGORITMOS

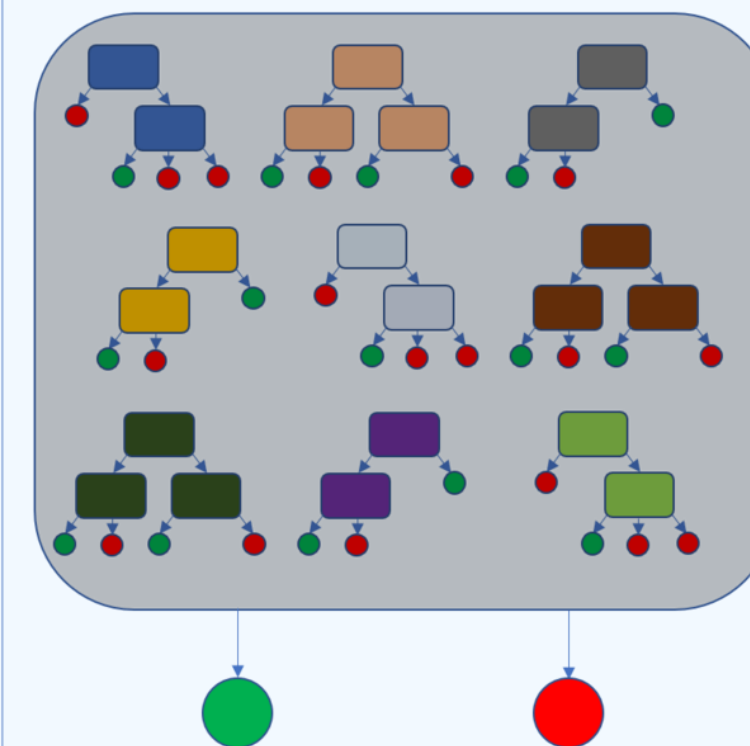
## – ENSEMBLE

### RANDOM FOREST

- **Pontos Negativos:**
  - Ainda mais "caixa-preta" que o Bagging.
  - Requer mais recursos computacionais e memória que um modelo único.



Decision Tree



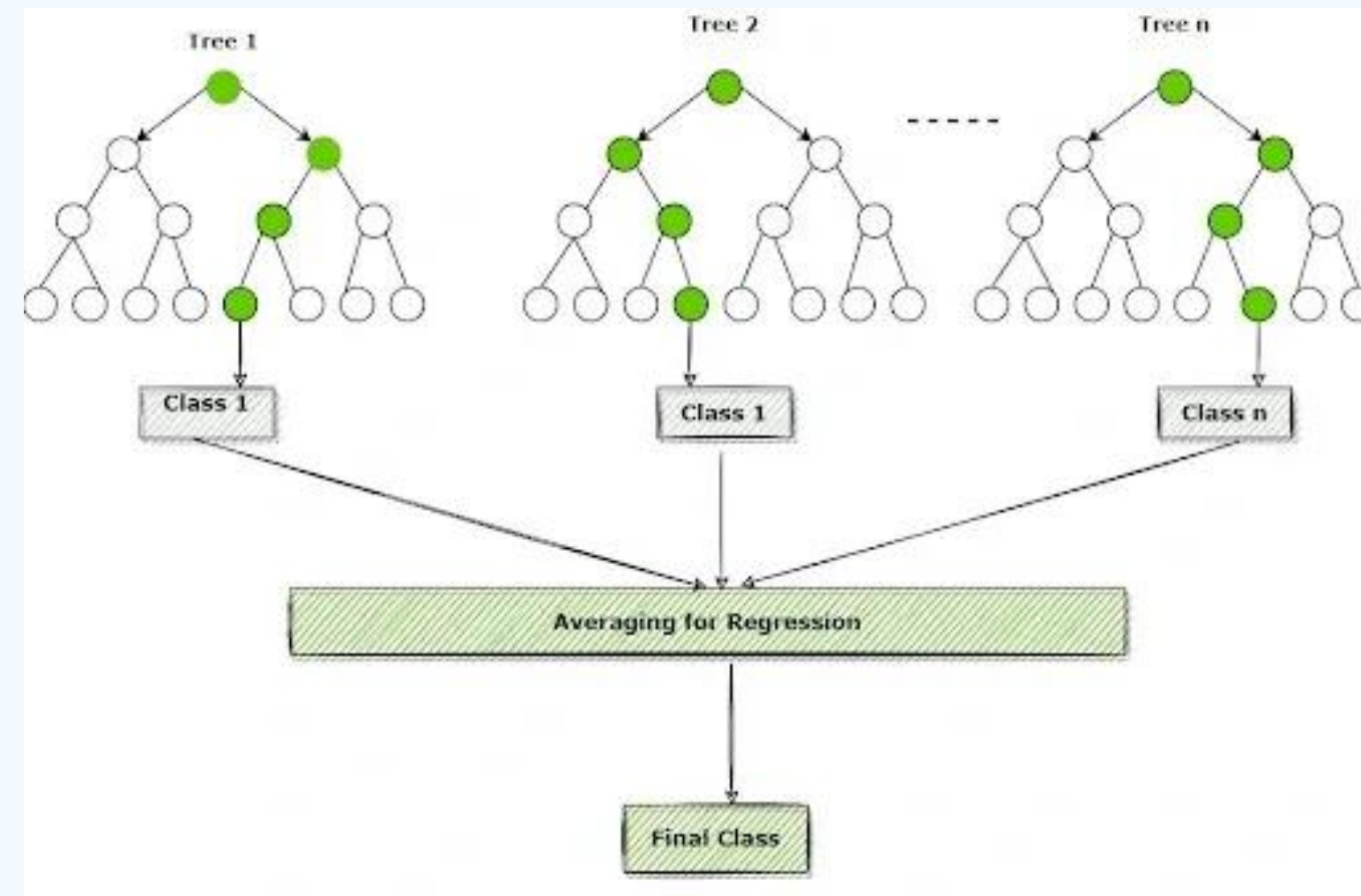
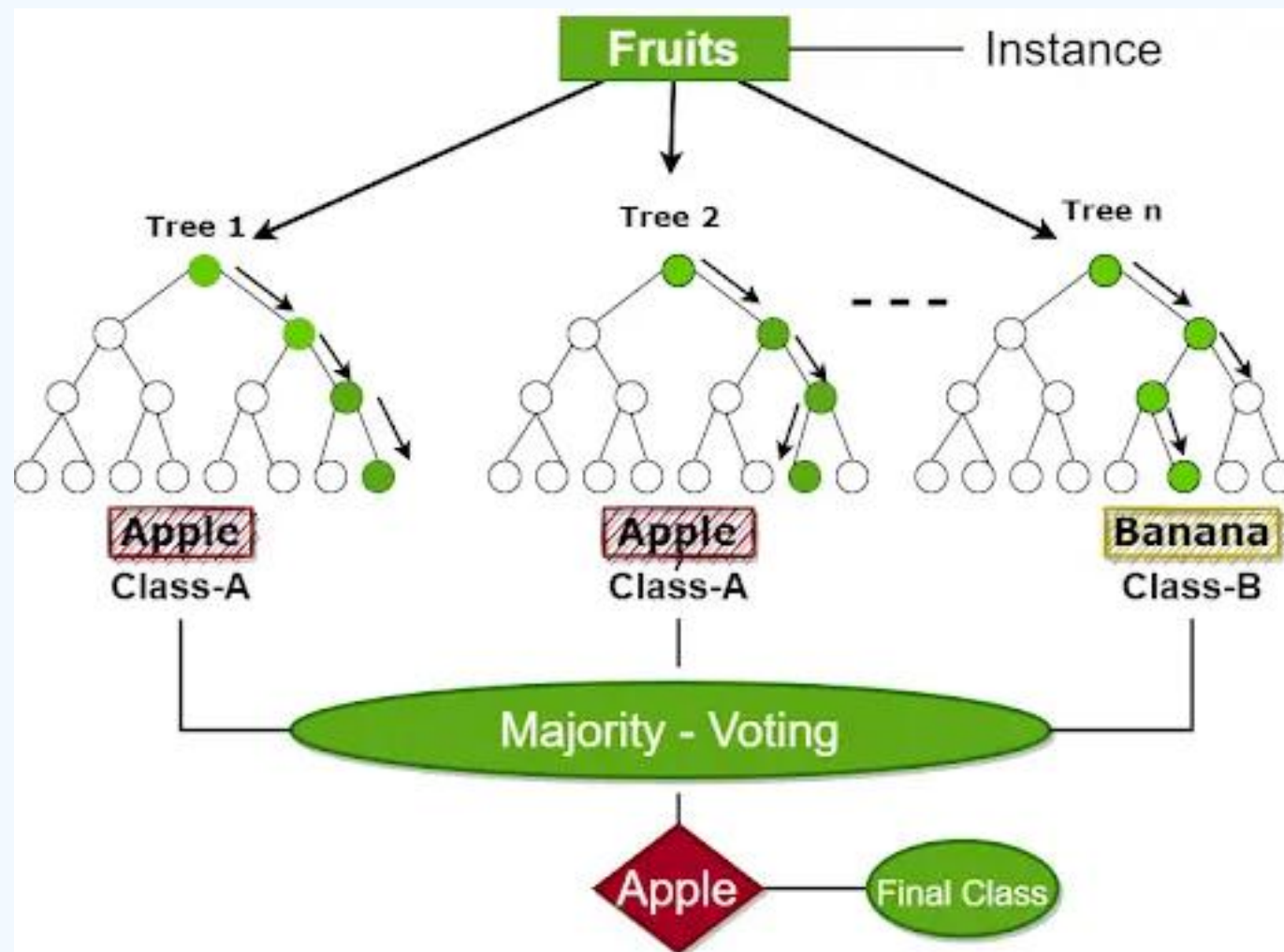
Random Forest

Fonte: <https://towardsmachinelearning.org/random-forest/>

# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

### RANDOM FOREST



# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

## RANDOM FOREST

Trabalho

sklearn.ensemble.RandomForest

+

https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html#sklearn.ensemble.RandomForestClassifier

learn

Install User Guide API Examples Community More

Prev Up Next

scikit-learn 1.3.2

Other versions

Please cite us if you use the software.

sklearn.ensemble.RandomForestClassifier

RandomForestClassifier

Examples using

sklearn.ensemble.RandomForestClassifier

sklearn.ensemble.RandomForestClassifier

```
class sklearn.ensemble.RandomForestClassifier(n_estimators=100, *, criterion='gini', max_depth=None, min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features='sqrt', max_leaf_nodes=None, min_impurity_decrease=0.0, bootstrap=True, oob_score=False, n_jobs=None, random_state=None, verbose=0, warm_start=False, class_weight=None, ccp_alpha=0.0, max_samples=None)
```

[source]

A random forest classifier.

A random forest is a meta estimator that fits a number of decision tree classifiers on various sub-samples of the dataset and uses averaging to improve the predictive accuracy and control over-fitting. The sub-sample size is controlled with the `max_samples` parameter if `bootstrap=True` (default), otherwise the whole dataset is used to build each tree.

For a comparison between tree-based ensemble models see the example [Comparing Random Forests and Histogram Gradient Boosting models](#).

Read more in the [User Guide](#).

Parameters:

**n\_estimators : int, default=100**  
The number of trees in the forest.

Changed in version 0.22: The default value of `n_estimators` changed from 10 to 100 in 0.22.

**criterion : {'gini', 'entropy', 'log\_loss'}, default='gini'**  
The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "log\_loss" and "entropy" both for the Shannon information gain, see [Mathematical formulation](#). Note: This parameter is tree-specific.

**max\_depth : int, default=None**  
The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than `min_samples_split` samples.

**min\_samples\_split : int or float, default=2**  
The minimum number of samples required to split an internal node:

- If int, then consider `min_samples_split` as the minimum number.
- If float, then `min_samples_split` is a fraction and `ceil(min_samples_split * n_samples)` are the minimum number of samples for each split.

Changed in version 0.18: Added float values for fractions.

**min\_samples\_leaf : int or float, default=1**  
The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least `min_samples_leaf` training samples in each of the left and right branches. This may have the effect of smoothing the model, especially in regression.

- If int, then consider `min_samples_leaf` as the minimum number.
- If float, then `min_samples_leaf` is a fraction and `ceil(min_samples_leaf * n_samples)` are the minimum number of samples for each node.

Changed in version 0.18: Added float values for fractions.

**min\_weight\_fraction\_leaf : float, default=0.0**  
The minimum weighted fraction of the sum total of weights (of all the input samples) required to be at a leaf

Trabalho

sklearn.ensemble.RandomForest

+

https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html#sklearn.ensemble.RandomForestRegressor

learn

Install User Guide API Examples Community More

Prev Up Next

scikit-learn 1.3.2

Other versions

Please cite us if you use the software.

sklearn.ensemble.RandomForestRegressor

RandomForestRegressor

Examples using

sklearn.ensemble.RandomForestRegressor

sklearn.ensemble.RandomForestRegressor

```
class sklearn.ensemble.RandomForestRegressor(n_estimators=100, *, criterion='squared_error', max_depth=None, min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features=1.0, max_leaf_nodes=None, min_impurity_decrease=0.0, bootstrap=True, oob_score=False, n_jobs=None, random_state=None, verbose=0, warm_start=False, ccp_alpha=0.0, max_samples=None)
```

[source]

A random forest regressor.

A random forest is a meta estimator that fits a number of classifying decision trees on various sub-samples of the dataset and uses averaging to improve the predictive accuracy and control over-fitting. The sub-sample size is controlled with the `max_samples` parameter if `bootstrap=True` (default), otherwise the whole dataset is used to build each tree.

For a comparison between tree-based ensemble models see the example [Comparing Random Forests and Histogram Gradient Boosting models](#).

Read more in the [User Guide](#).

Parameters:

**n\_estimators : int, default=100**  
The number of trees in the forest.

Changed in version 0.22: The default value of `n_estimators` changed from 10 to 100 in 0.22.

**criterion : {'squared\_error', 'absolute\_error', 'friedman\_mse', 'poisson'}, default='squared\_error'**  
The function to measure the quality of a split. Supported criteria are "squared\_error" for the mean squared error, which is equal to variance reduction as feature selection criterion and minimizes the L2 loss using the mean of each terminal node, "friedman\_mse", which uses mean squared error with Friedman's improvement score for potential splits, "absolute\_error" for the mean absolute error, which minimizes the L1 loss using the median of each terminal node, and "poisson" which uses reduction in Poisson deviance to find splits. Training using "absolute\_error" is significantly slower than when using "squared\_error".

New in version 0.18: Mean Absolute Error (MAE) criterion.

New in version 1.0: Poisson criterion.

**max\_depth : int, default=None**  
The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than `min_samples_split` samples.

**min\_samples\_split : int or float, default=2**  
The minimum number of samples required to split an internal node:

- If int, then consider `min_samples_split` as the minimum number.
- If float, then `min_samples_split` is a fraction and `ceil(min_samples_split * n_samples)` are the minimum number of samples for each split.

Changed in version 0.18: Added float values for fractions.

**min\_samples\_leaf : int or float, default=1**  
The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least `min_samples_leaf` training samples in each of the left and right branches. This may have the effect of smoothing the model, especially in regression.

- If int, then consider `min_samples_leaf` as the minimum number.

# TÓPICO 3

**BOOSTING: GRADIENT  
BOOSTING, XGBOOST,  
LIGHTGBM E CATBOOST**

# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

### ADABOOST

- O primeiro algoritmo de Boosting de grande sucesso. Ele ajusta o peso das amostras de treinamento a cada iteração, forçando os modelos subsequentes a se concentrarem nas observações mais difíceis de classificar.
- **Funcionamento:**
  - **Inicialização:** Todas as  $N$  amostras recebem um peso igual  $w_i = \frac{1}{N}$ .
  - **Iteração (para  $m = 1$  até  $M$ ):**
    - Treine um modelo fraco ( $h_m$ ) que minimize o erro ponderado pelos pesos  $w_i$ .
    - Calcule o erro do modelo:  $\varepsilon_m = \sum_{i \text{ class. incorretamente}} w_i$ .
    - Calcule o peso do modelo na votação final:  $\alpha_m = \frac{1}{2} * \ln \left( \frac{(1 - \varepsilon_m)}{\varepsilon_m} \right)$ .
    - Atualize os pesos das amostras  $w_i$ , aumentando o peso dos erros.
- **Predição Final:** A predição é a soma ponderada das predições de todos os modelos:  $H(x) = \text{sign}(\sum_{m=1}^M \alpha_m * h_{m(x)})$

# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

### ADABOOST

- **Pontos Positivos:**
  - Alto desempenho e fácil de entender.
  - Não requer muitos ajustes de hiperparâmetros.
- **Pontos Negativos:**
  - Sensível a ruído e outliers, pois pode se concentrar demais neles.
  - Processo sequencial, não pode ser paralelizado.
  - Superado por algoritmos mais modernos como o Gradient Boosting.

# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

### ADABOOST

Trabalho

adaboost - Pesquisar

sklearn.ensemble.AdaBoostClassifier

https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.AdaBoostClassifier.html

Install User Guide API Examples Community More

Prev Up Next

scikit-learn 1.3.2  
Other versions

Please cite us if you use the software.

sklearn.ensemble.AdaBoostClassifier

AdaBoostClassifier

Examples using sklearn.ensemble.AdaBoostClassifier

sklearn.ensemble.AdaBoostClassifier

```
class sklearn.ensemble.AdaBoostClassifier(estimator=None, *, n_estimators=50, learning_rate=1.0, algorithm='SAMME.R', random_state=None, base_estimator='deprecated') [source]
```

An AdaBoost classifier.

An AdaBoost [1] classifier is a meta-estimator that begins by fitting a classifier on the original dataset and then fits additional copies of the classifier on the same dataset but where the weights of incorrectly classified instances are adjusted such that subsequent classifiers focus more on difficult cases.

This class implements the algorithm known as AdaBoost-SAMME [2].

Read more in the User Guide.

New in version 0.14.

Parameters:

**estimator : object, default=None**  
The base estimator from which the boosted ensemble is built. Support for sample weighting is required, as well as proper `classes_` and `n_classes_` attributes. If `None`, then the base estimator is `DecisionTreeClassifier` initialized with `max_depth=1`.

New in version 1.2: `base_estimator` was renamed to `estimator`.

**n\_estimators : int, default=50**  
The maximum number of estimators at which boosting is terminated. In case of perfect fit, the learning procedure is stopped early. Values must be in the range `[1, inf)`.

**learning\_rate : float, default=1.0**  
Weight applied to each classifier at each boosting iteration. A higher learning rate increases the contribution of each classifier. There is a trade-off between the `learning_rate` and `n_estimators` parameters. Values must be in the range `(0.0, inf)`.

**algorithm : ('SAMME', 'SAMME.R'), default='SAMME.R'**  
If 'SAMME.R' then use the SAMME.R real boosting algorithm. `estimator` must support calculation of class probabilities. If 'SAMME' then use the SAMME discrete boosting algorithm. The SAMME.R algorithm typically converges faster than SAMME, achieving a lower test error with fewer boosting iterations.

**random\_state : int, RandomState instance or None, default=None**  
Controls the random seed given at each `estimator` at each boosting iteration. Thus, it is only used when `estimator` exposes a `random_state`. Pass an int for reproducible output across multiple function calls. See Glossary.

**base\_estimator : object, default=None**  
The base estimator from which the boosted ensemble is built. Support for sample weighting is required, as well as proper `classes_` and `n_classes_` attributes. If `None`, then the base estimator is `DecisionTreeClassifier` initialized with `max_depth=1`.

Deprecated since version 1.2: `base_estimator` is deprecated and will be removed in 1.4. Use `estimator` instead.

Attributes:

**estimator\_ : estimator**  
The base estimator from which the ensemble is grown.

Trabalho

adaboost - Pesquisar

sklearn.ensemble.AdaBoostRegressor

https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.AdaBoostRegressor.html#sklearn.ensemble.AdaBoostRegressor

Install User Guide API Examples Community More

Prev Up Next

scikit-learn 1.3.2  
Other versions

Please cite us if you use the software.

sklearn.ensemble.AdaBoostRegressor

AdaBoostRegressor

Examples using sklearn.ensemble.AdaBoostRegressor

sklearn.ensemble.AdaBoostRegressor

```
class sklearn.ensemble.AdaBoostRegressor(estimator=None, *, n_estimators=50, learning_rate=1.0, loss='linear', random_state=None, base_estimator='deprecated') [source]
```

An AdaBoost regressor.

An AdaBoost [1] regressor is a meta-estimator that begins by fitting a regressor on the original dataset and then fits additional copies of the regressor on the same dataset but where the weights of instances are adjusted according to the error of the current prediction. As such, subsequent regressors focus more on difficult cases.

This class implements the algorithm known as AdaBoost.R2 [2].

Read more in the User Guide.

New in version 0.14.

Parameters:

**estimator : object, default=None**  
The base estimator from which the boosted ensemble is built. If `None`, then the base estimator is `DecisionTreeRegressor` initialized with `max_depth=3`.

New in version 1.2: `base_estimator` was renamed to `estimator`.

**n\_estimators : int, default=50**  
The maximum number of estimators at which boosting is terminated. In case of perfect fit, the learning procedure is stopped early. Values must be in the range `[1, inf)`.

**learning\_rate : float, default=1.0**  
Weight applied to each regressor at each boosting iteration. A higher learning rate increases the contribution of each regressor. There is a trade-off between the `learning_rate` and `n_estimators` parameters. Values must be in the range `(0.0, inf)`.

**loss : ('linear', 'square', 'exponential'), default='linear'**  
The loss function to use when updating the weights after each boosting iteration.

**random\_state : int, RandomState instance or None, default=None**  
Controls the random seed given at each `estimator` at each boosting iteration. Thus, it is only used when `estimator` exposes a `random_state`. In addition, it controls the bootstrap of the weights used to train the `estimator` at each boosting iteration. Pass an int for reproducible output across multiple function calls. See Glossary.

**base\_estimator : object, default=None**  
The base estimator from which the boosted ensemble is built. If `None`, then the base estimator is `DecisionTreeRegressor` initialized with `max_depth=3`.

Deprecated since version 1.2: `base_estimator` is deprecated and will be removed in 1.4. Use `estimator` instead.

Attributes:

**estimator\_ : estimator**  
The base estimator from which the ensemble is grown.

New in version 1.2: `base_estimator_` was renamed to `estimator_`.

**base\_estimator\_ : estimator**  
Estimator used to grow the ensemble

# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

### BOOSTING – GBM

- **Conceito Central:** Construir um modelo forte a partir de uma sequência de modelos fracos. Diferente do Bagging, o Boosting é um processo sequencial, onde cada novo modelo é treinado para corrigir os erros cometidos pelos modelos anteriores.
- Em vez de ajustar os pesos das amostras, o GBM treina sequencialmente novos modelos para prever os resíduos (os erros) do modelo anterior. Ele trata o problema como uma otimização numérica, usando a descida de gradiente para minimizar uma função de perda arbitrária.
- **Funcionamento:**
  - O GBM minimiza uma função de perda  $L(y, F(x))$  usando descida de gradiente no espaço das funções. O "gradiente" é o resíduo e, a cada passo, o algoritmo move-se na direção do gradiente negativo (o erro) para minimizar a perda geral. A flexibilidade de escolher qualquer função de perda diferenciável é sua grande vantagem.

# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

### BOOSTING – GBM

- Exemplo:

- Problema: Prever o salário de uma pessoa.
- Passo 1: A primeira predição para todos é o salário médio: R\$ 4.000.
- Passo 2: Calcule os erros. Pessoa A (salário real R\$ 5.000) tem um erro de +R\$ 1.000. Pessoa B (salário real R\$ 3.500) tem um erro de -R\$ 500.
- Passo 3: Treine uma árvore para prever esses erros (+1000, -500, etc.) com base nas features.
- Passo 4: A nova predição será: Predição Anterior + (taxa\_aprendizado \* predição\_de\_erro).
- Passo 5: O processo se repete, refinando os erros a cada passo.

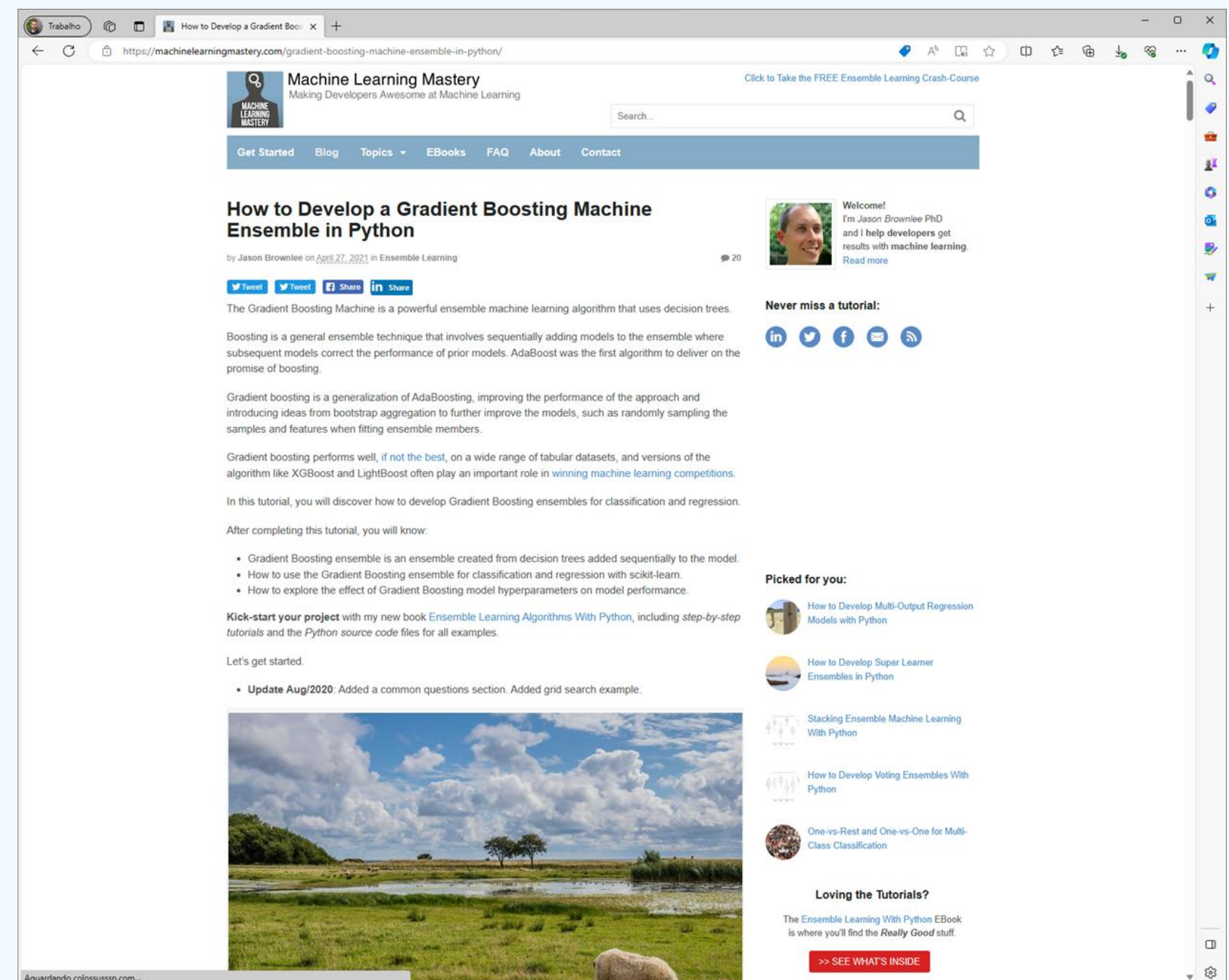
# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

## BOOSTING – GBM

```
1 # evaluate gradient boosting algorithm for classification
2 from numpy import mean
3 from numpy import std
4 from sklearn.datasets import make_classification
5 from sklearn.model_selection import cross_val_score
6 from sklearn.model_selection import RepeatedStratifiedKFold
7 from sklearn.ensemble import GradientBoostingClassifier
8 # define dataset
9 X, y = make_classification(n_samples=1000, n_features=20, n_informative=15, n_redundant=5, random_state=7)
10 # define the model
11 model = GradientBoostingClassifier()
12 # define the evaluation method
13 cv = RepeatedStratifiedKFold(n_splits=10, n_repeats=3, random_state=1)
14 # evaluate the model on the dataset
15 n_scores = cross_val_score(model, X, y, scoring='accuracy', cv=cv, n_jobs=-1)
16 # report performance
17 print('Mean Accuracy: %.3f (%.3f)' % (mean(n_scores), std(n_scores)))
```

```
1 # make predictions using gradient boosting for classification
2 from sklearn.datasets import make_classification
3 from sklearn.ensemble import GradientBoostingClassifier
4 # define dataset
5 X, y = make_classification(n_samples=1000, n_features=20, n_informative=15, n_redundant=5, random_state=7)
6 # define the model
7 model = GradientBoostingClassifier()
8 # fit the model on the whole dataset
9 model.fit(X, y)
10 # make a single prediction
11 row = [0.2929949, -4.21223056, -1.288332, -2.17849815, -0.64527665, 2.58097719, 0.28422388, -7.1827928, -1.91211104, 2.73729512, 0.81395695, 3.96973717, -2.66939799, 3.34692332, 4.19791821, 0.99990998, -0.30201875, -4.43170633, -2.82646737, 0.44916808]
12 yhat = model.predict([row])
13 # summarize prediction
14 print('Predicted Class: %d' % yhat[0])
```



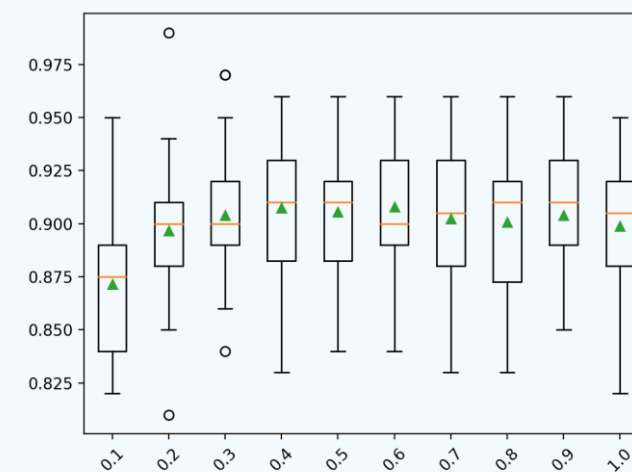
The screenshot shows a web browser displaying the Machine Learning Mastery website. The page title is "How to Develop a Gradient Boosting Machine Ensemble in Python". The author is Jason Brownlee, PhD, and the article was published on April 27, 2021. The page includes a search bar, navigation links (Get Started, Blog, Topics, EBooks, FAQ, About, Contact), and a sidebar with social media links and a "Never miss a tutorial" section. The main content area discusses the Gradient Boosting Machine (GBM) algorithm, its generalization of AdaBoost, and its application in ensemble learning. It mentions that GBM is a powerful ensemble machine learning algorithm that uses decision trees. The article also includes a "Kick-start your project" section with a link to the author's book "Ensemble Learning Algorithms With Python". The page features a large image of a landscape with a body of water and trees under a blue sky with clouds.

# PRINCIPAIS ALGORITMOS

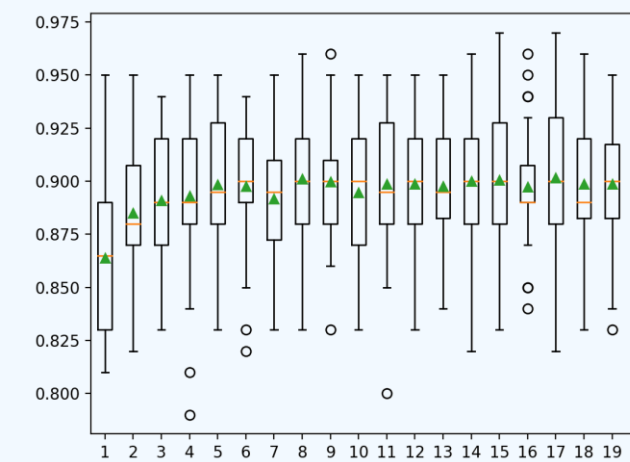
## – ENSEMBLE

### BOOSTING – GBM

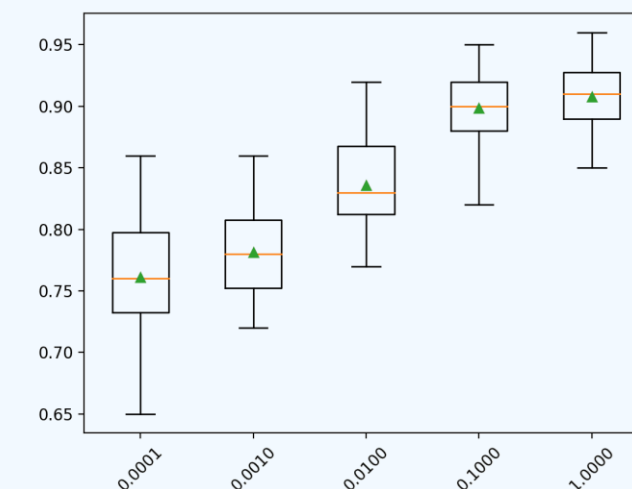
**Tamanho da  
Amostra**



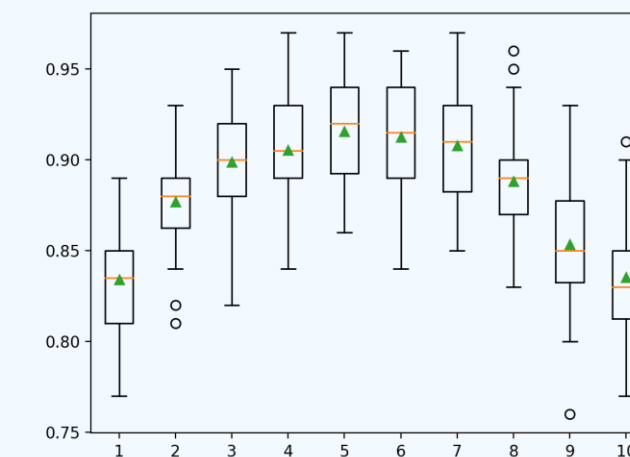
**Número de  
Atributos**



**Taxa de  
Aprendizado**



**Profundidade da  
Árvore**



# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

## BOOSTING – GBM

Trabalho

sklearn.ensemble.GradientBoos

+

https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingClassifier.html#sklearn.ensemble.GradientBoostingClassifier

learn

Install User Guide API Examples Community More

Go

Prev Up Next

scikit-learn 1.3.2

Other versions

Please cite us if you use the software.

sklearn.ensemble.GradientBoos

tingClassifier

GradientBoostingClassifier

Examples using

sklearn.ensemble.GradientBoosting

sklearn.ensemble.GradientBoostingClassifier

```
class sklearn.ensemble.GradientBoostingClassifier(*, loss='log_loss', learning_rate=0.1, n_estimators=100, subsample=1.0, criterion='friedman_mse', min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_depth=3, min_impurity_decrease=0.0, init=None, random_state=None, max_features=None, verbose=0, max_leaf_nodes=None, warm_start=False, validation_fraction=0.1, n_iter_no_change=None, tol=0.0001, ccp_alpha=0.0)
```

Gradient Boosting for classification.

This algorithm builds an additive model in a forward stage-wise fashion; it allows for the optimization of arbitrary differentiable loss functions. In each stage `n_classes` regression trees are fit on the negative gradient of the loss function, e.g. binary or multiclass log loss. Binary classification is a special case where only a single regression tree is induced.

`sklearn.ensemble.HistGradientBoostingClassifier` is a much faster variant of this algorithm for intermediate datasets (`n_samples`  $\geq 10,000$ ).

Read more in the [User Guide](#).

Parameters:

**loss** : ('log\_loss', 'exponential'), default='log\_loss'

The loss function to be optimized. 'log\_loss' refers to binomial and multinomial deviance, the same as used in logistic regression. It is a good choice for classification with probabilistic outputs. For loss 'exponential', gradient boosting recovers the AdaBoost algorithm.

**learning\_rate** : float, default=0.1

Learning rate shrinks the contribution of each tree by `learning_rate`. There is a trade-off between learning\_rate and `n_estimators`. Values must be in the range `[0.0, inf)`.

**n\_estimators** : int, default=100

The number of boosting stages to perform. Gradient boosting is fairly robust to over-fitting so a large number usually results in better performance. Values must be in the range `[1, inf)`.

**subsample** : float, default=1.0

The fraction of samples to be used for fitting the individual base learners. If smaller than 1.0 this results in Stochastic Gradient Boosting. `subsample` interacts with the parameter `n_estimators`. Choosing `subsample`  $< 1.0$  leads to a reduction of variance and an increase in bias. Values must be in the range `(0.0, 1.0]`.

**criterion** : ('friedman\_mse', 'squared\_error'), default='friedman\_mse'

The function to measure the quality of a split. Supported criteria are 'friedman\_mse' for the mean squared error with improvement score by Friedman, 'squared\_error' for mean squared error. The default value of 'friedman\_mse' is generally the best as it can provide a better approximation in some cases.

*New in version 0.18.*

**min\_samples\_split** : int or float, default=2

The minimum number of samples required to split an internal node:

- If int, values must be in the range `[2, inf)`.
- If float, values must be in the range `(0.0, 1.0]` and `min_samples_split` will be `ceil(min_samples_split * n_samples)`.

*Changed in version 0.18: Added float values for fractions.*

**min\_samples\_leaf** : int or float, default=1

The minimum number of samples required to be at a leaf node. A split point at any depth will only be

Trabalho

sklearn.ensemble.GradientBoos

+

https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingRegressor.html#sklearn.ensemble.GradientBoostingRegressor

learn

Install User Guide API Examples Community More

Go

Prev Up Next

scikit-learn 1.3.2

Other versions

Please cite us if you use the software.

sklearn.ensemble.GradientBoos

tingRegressor

GradientBoostingRegressor

Examples using

sklearn.ensemble.GradientBoosting

sklearn.ensemble.GradientBoostingRegressor

```
class sklearn.ensemble.GradientBoostingRegressor(*, loss='squared_error', learning_rate=0.1, n_estimators=100, subsample=1.0, criterion='friedman_mse', min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_depth=3, min_impurity_decrease=0.0, init=None, random_state=None, max_features=None, alpha=0.9, verbose=0, max_leaf_nodes=None, warm_start=False, validation_fraction=0.1, n_iter_no_change=None, tol=0.0001, ccp_alpha=0.0)
```

Gradient Boosting for regression.

This estimator builds an additive model in a forward stage-wise fashion; it allows for the optimization of arbitrary differentiable loss functions. In each stage a regression tree is fit on the negative gradient of the given loss function.

`sklearn.ensemble.HistGradientBoostingRegressor` is a much faster variant of this algorithm for intermediate datasets (`n_samples`  $\geq 10,000$ ).

Read more in the [User Guide](#).

Parameters:

**loss** : ('squared\_error', 'absolute\_error', 'huber', 'quantile'), default='squared\_error'

Loss function to be optimized. 'squared\_error' refers to the squared error for regression. 'absolute\_error' refers to the absolute error of regression and is a robust loss function. 'huber' is a combination of the two. 'quantile' allows quantile regression (use `alpha` to specify the quantile).

**learning\_rate** : float, default=0.1

Learning rate shrinks the contribution of each tree by `learning_rate`. There is a trade-off between learning\_rate and `n_estimators`. Values must be in the range `[0.0, inf)`.

**n\_estimators** : int, default=100

The number of boosting stages to perform. Gradient boosting is fairly robust to over-fitting so a large number usually results in better performance. Values must be in the range `[1, inf)`.

**subsample** : float, default=1.0

The fraction of samples to be used for fitting the individual base learners. If smaller than 1.0 this results in Stochastic Gradient Boosting. `subsample` interacts with the parameter `n_estimators`. Choosing `subsample`  $< 1.0$  leads to a reduction of variance and an increase in bias. Values must be in the range `(0.0, 1.0]`.

**criterion** : ('friedman\_mse', 'squared\_error'), default='friedman\_mse'

The function to measure the quality of a split. Supported criteria are 'friedman\_mse' for the mean squared error with improvement score by Friedman, 'squared\_error' for mean squared error. The default value of 'friedman\_mse' is generally the best as it can provide a better approximation in some cases.

*New in version 0.18.*

**min\_samples\_split** : int or float, default=2

The minimum number of samples required to split an internal node:

- If int, values must be in the range `[2, inf)`.
- If float, values must be in the range `(0.0, 1.0]` and `min_samples_split` will be `ceil(min_samples_split * n_samples)`.

*Changed in version 0.18: Added float values for fractions.*

**min\_samples\_leaf** : int or float, default=1

The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least `min_samples_leaf` training samples in each of the left and right branches. This

# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

### XGBOOST

- Uma implementação otimizada e regularizada do GBM que se tornou o padrão ouro para dados tabulares.
- Principais melhorias:
  - **Regularização:** Adiciona penalidades (L1 e L2) na função de perda para controlar a complexidade das árvores e evitar *overfitting*.
  - **Otimização de Gradiente:** Usa a expansão de Taylor de segunda ordem (Hessiana) para um passo de otimização mais preciso.
  - **Eficiência Computacional:** Altamente otimizado com paralelização, tratamento de dados esparsos e algoritmos eficientes para encontrar divisões.

# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

### LGBM

- Uma implementação do GBM focada em velocidade e eficiência de memória.
- Principais melhorias:
  - **Crescimento Leaf-wise:** Em vez de crescer a árvore nível por nível (como o XGBoost), ele expande a folha que trará a maior redução de perda. Isso é mais rápido, mas pode levar a overfitting em datasets pequenos.
  - **Técnicas de Amostragem (GOSS & EFB):** Usa métodos inteligentes para amostrar dados e features, acelerando drasticamente o treinamento sem grande perda de precisão.

# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

### XGBOOST/LGBM

- **Pontos Positivos:**

- Performance de ponta na maioria dos problemas com dados tabulares.
- Grande flexibilidade (funções de perda customizadas).
- Controle muito robusto de *overfitting*, especialmente no XGBoost.

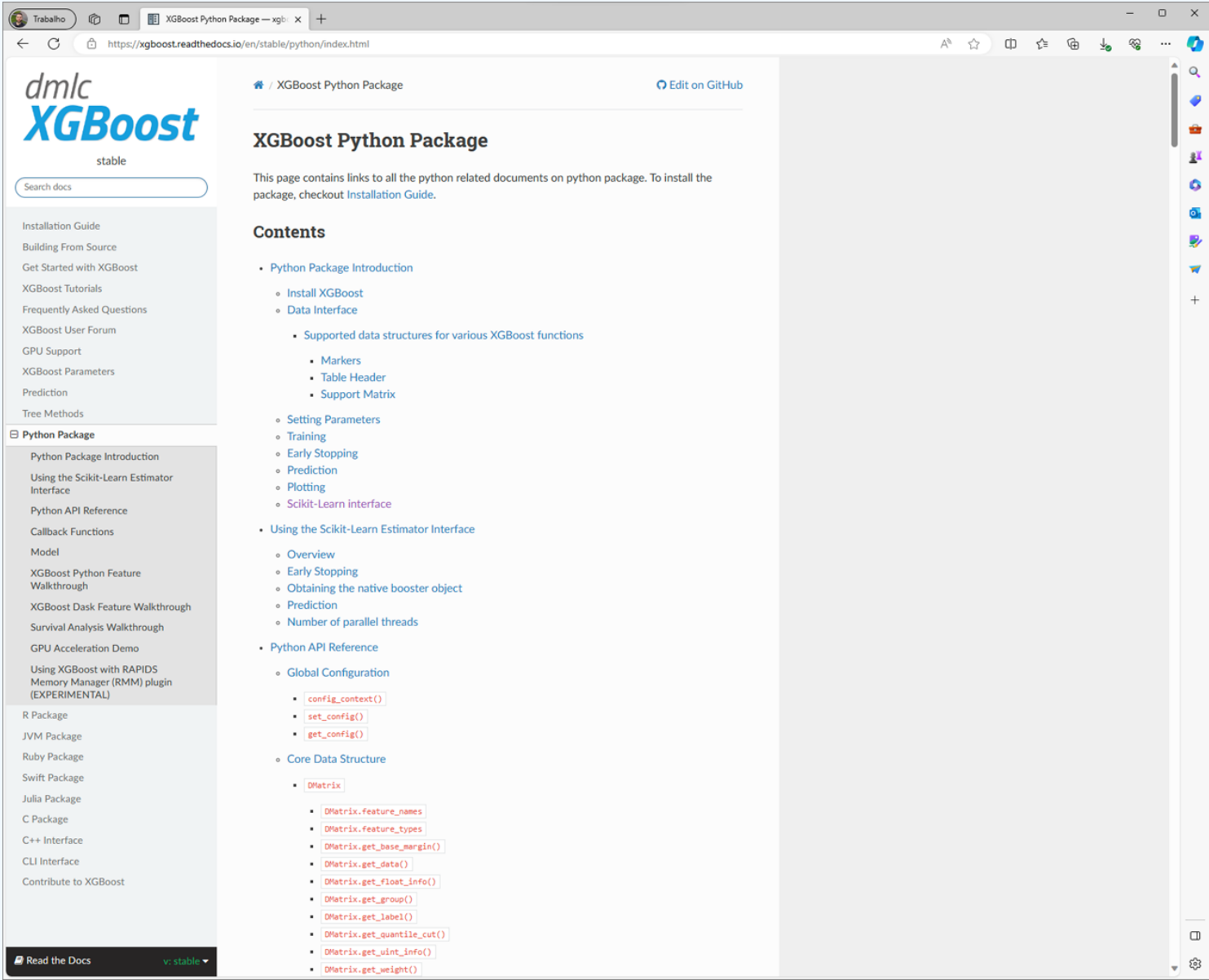
- **Pontos Negativos:**

- Muitos hiperparâmetros para ajustar, exigindo mais experiência.
- Ainda é um processo sequencial e pode ser computacionalmente caro.
- Altamente "caixa-preta".

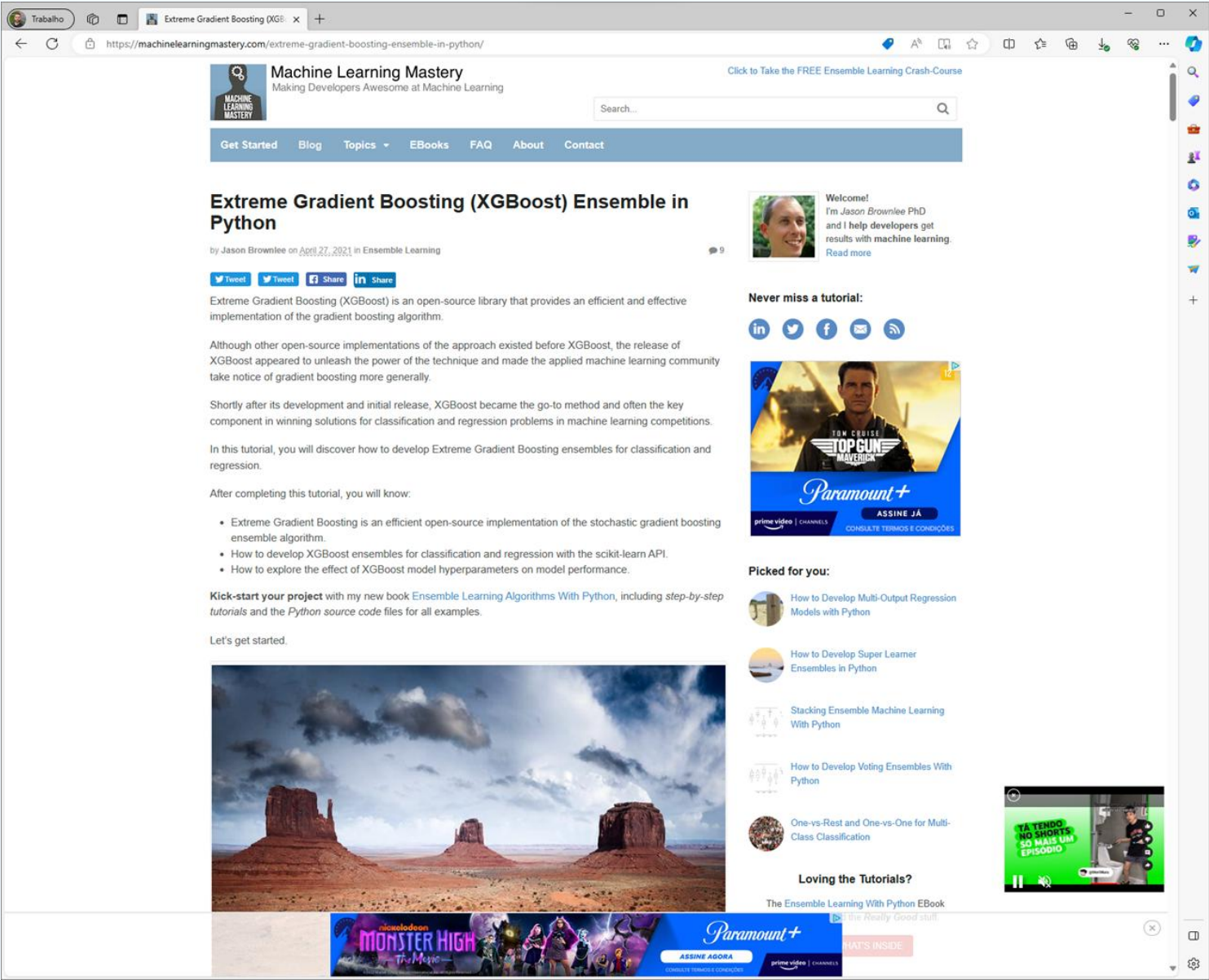
# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

## XGBOOST



The screenshot shows the official XGBoost Python Package documentation page. The page title is "XGBoost Python Package" and it includes a link to "Edit on GitHub". The main content area is titled "XGBoost Python Package" and contains a brief introduction: "This page contains links to all the python related documents on python package. To install the package, checkout Installation Guide." Below this is a "Contents" section with a list of links: "Python Package Introduction", "Install XGBoost", "Data Interface", "Supported data structures for various XGBoost functions" (with sub-links for Markers, Table Header, and Support Matrix), "Setting Parameters", "Training", "Early Stopping", "Prediction", "Plotting", "Scikit-Learn interface", "Using the Scikit-Learn Estimator Interface", "Overview", "Early Stopping", "Obtaining the native booster object", "Prediction", "Number of parallel threads", "Python API Reference", "Global Configuration" (with sub-links for config\_context(), set\_config(), and get\_config()), and "Core Data Structure" (with a sub-link for DMatrix). The DMatrix section lists various methods like feature\_names, feature\_types, get\_base\_margin(), get\_data(), get\_float\_info(), get\_group(), get\_label(), get\_quantile\_cut(), get\_uint\_info(), and get\_weight(). A sidebar on the left contains a search bar and a list of navigation links including "Installation Guide", "Building From Source", "Get Started with XGBoost", "XGBoost Tutorials", "Frequently Asked Questions", "XGBoost User Forum", "GPU Support", "XGBoost Parameters", "Prediction", "Tree Methods", "Python Package", "Python Package Introduction", "Using the Scikit-Learn Estimator Interface", "Python API Reference", "Callback Functions", "Model", "XGBoost Python Feature Walkthrough", "XGBoost Dask Feature Walkthrough", "Survival Analysis Walkthrough", "GPU Acceleration Demo", "Using XGBoost with RAPIDS Memory Manager (RMM) plugin (EXPERIMENTAL)", "R Package", "JVM Package", "Ruby Package", "Swift Package", "Julia Package", "C Package", "C++ Interface", "CLI Interface", and "Contribute to XGBoost".

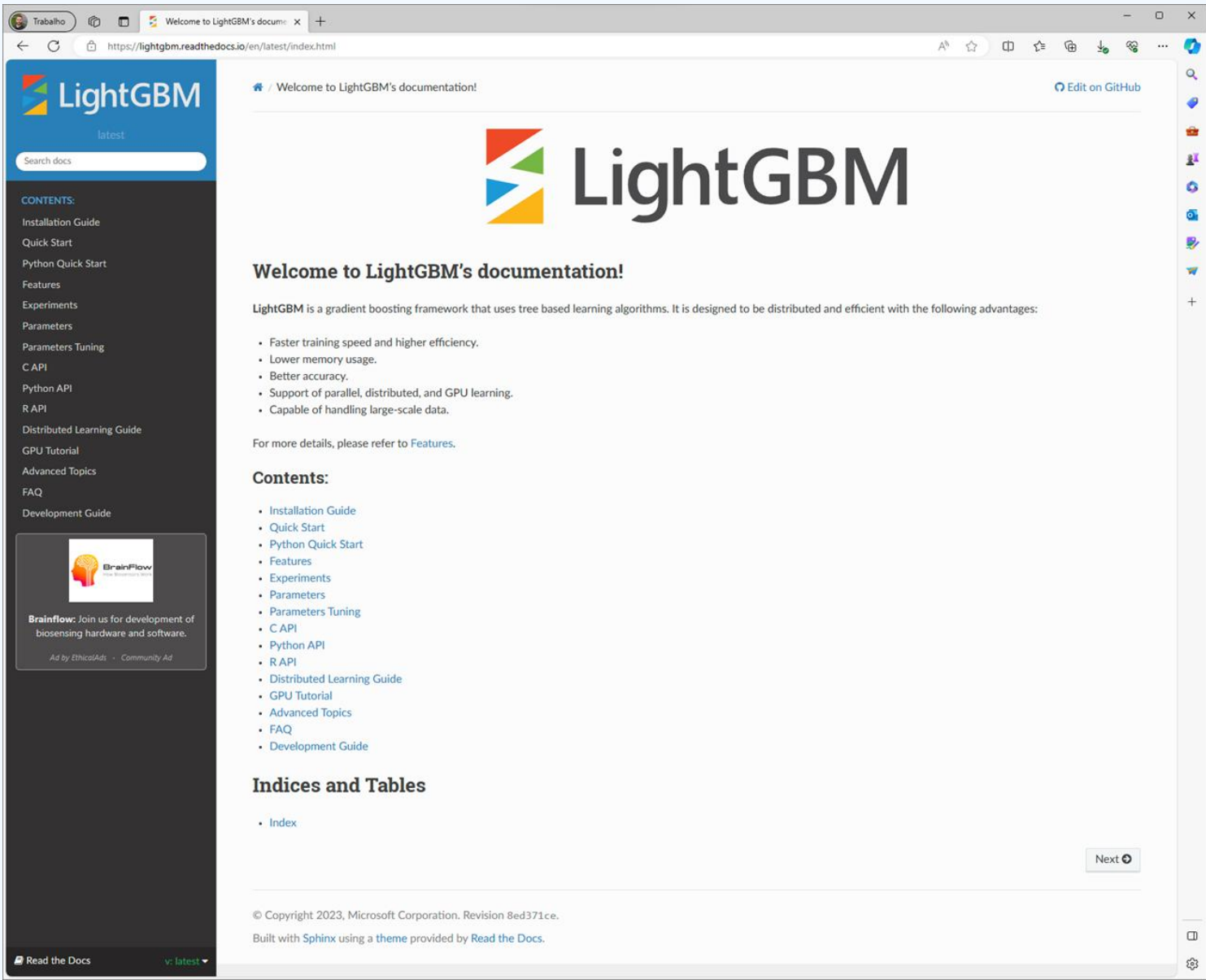


The screenshot shows an article titled "Extreme Gradient Boosting (XGBoost) Ensemble in Python" from the Machine Learning Mastery website. The article is by Jason Brownlee and was published on April 27, 2021. The article text describes XGBoost as an open-source library that provides an efficient and effective implementation of the gradient boosting algorithm. It mentions that XGBoost became the go-to method for classification and regression problems in machine learning competitions. The article also includes a list of "Picked for you:" recommendations, such as "How to Develop Multi-Output Regression Models with Python", "How to Develop Super Learner Ensembles in Python", "Stacking Ensemble Machine Learning With Python", "How to Develop Voting Ensembles With Python", and "One-vs-Rest and One-vs-One for Multi-Class Classification". The article also features a "Kick-start your project" section with a link to the book "Ensemble Learning Algorithms With Python". The article is accompanied by a large image of a desert landscape with rock formations under a cloudy sky. The page also includes a sidebar with a search bar and a list of navigation links including "Get Started", "Blog", "Topics", "EBooks", "FAQ", "About", and "Contact".

# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

## LGBM



The screenshot shows the official LightGBM documentation page. The header includes the LightGBM logo and a search bar. A left sidebar lists various content categories like 'Installation Guide', 'Quick Start', and 'Parameters'. The main content area welcomes users and lists key advantages of the framework, such as faster training speed and higher efficiency. It also provides a 'Contents' section with links to specific guides and an 'Indices and Tables' section with an 'Index' link.



The screenshot shows the Machine Learning Mastery website. The header features the site's name and a search bar. The main content area is titled 'How to Develop a Light Gradient Boosted Machine (LightGBM) Ensemble' and includes a brief introduction to the framework. A right sidebar offers additional resources, including a 'Never miss a tutorial' section with social media links and a 'Picked for you' section with links to related tutorials. The bottom of the page features a large image of a desert landscape and a video player.

# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

### CATBOOST

- Implementação de boosting machine especializada no tratamento nativo de variáveis categóricas — o ponto fraco histórico do GBM, XGBoost e LGBM, que exigem codificação prévia dos dados.
- **Melhorias:**
  - **Tratamento nativo de categóricas:** aplica *Ordered Target Statistics*, substituindo cada categoria por uma estatística da variável-alvo calculada apenas com as observações anteriores, evitando o vazamento de informação (*target leakage*) do target encoding tradicional.
  - **Ordered Boosting:** em vez de calcular os resíduos com o mesmo conjunto usado no treino, usa permutações aleatórias dos dados para estimar os gradientes, reduzindo o viés de predição (*prediction shift*).
  - **Árvores simétricas (*oblivious trees*):** todos os nós de um mesmo nível usam a mesma condição de divisão, o que funciona como regularização natural e torna a inferência extremamente rápida.
  - **Combinações de features:** gera automaticamente cruzamentos entre variáveis categóricas durante a construção das árvores.

# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

### CATBOOST

- **Pontos Positivos:**

- Excelente desempenho com dados categóricos, sem necessidade de pré-processamento manual.
- Muito bom com hiperparâmetros padrão, exigindo menos tuning que XGBoost e LGBM.
- Ordered Boosting reduz overfitting e o viés de predição em bases pequenas.
- Predição muito rápida graças às árvores simétricas.

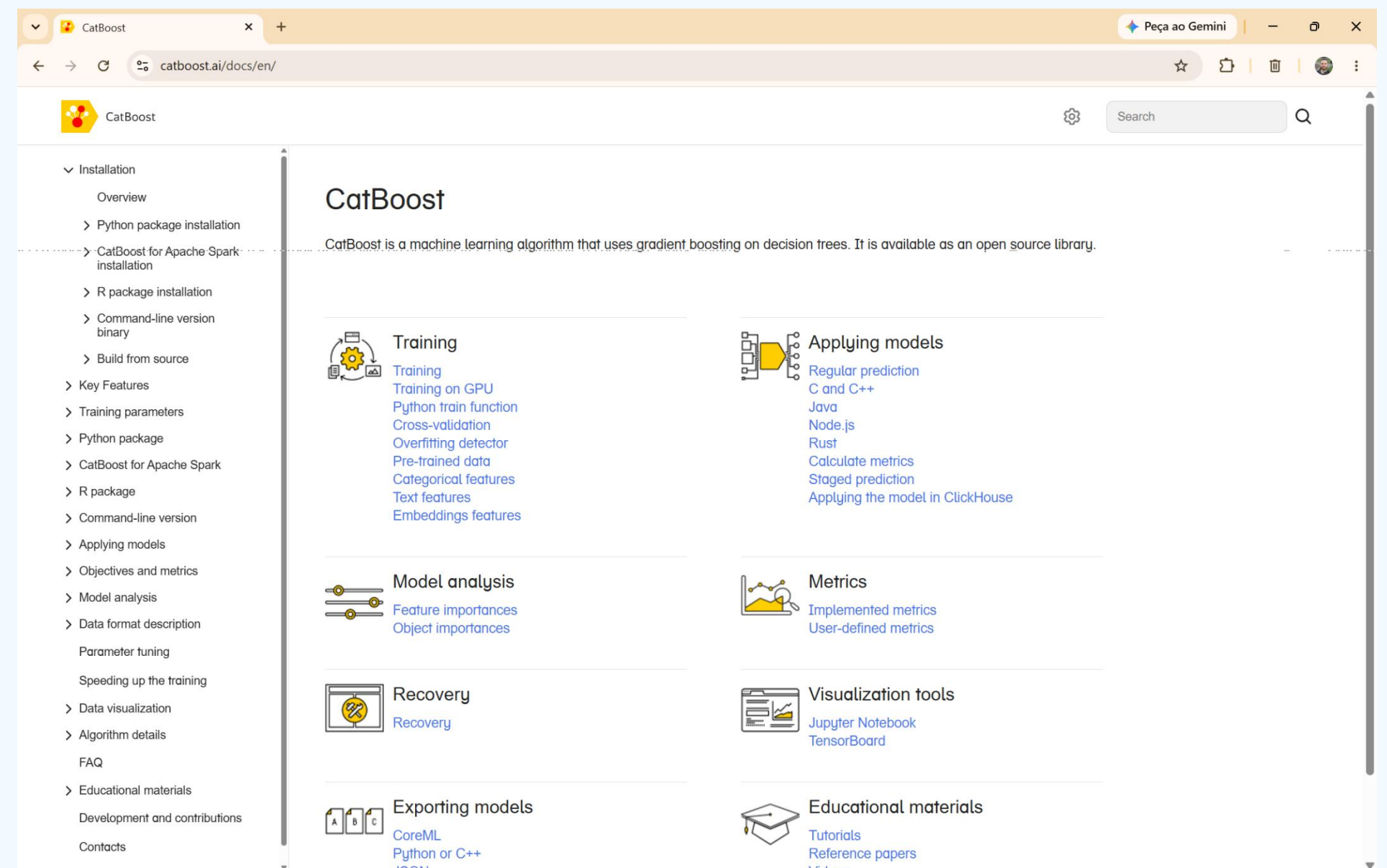
- **Pontos Negativos:**

- Treinamento mais lento que o LGBM em bases grandes e majoritariamente numéricas.
- Maior consumo de memória por causa das permutações e das combinações de categorias.
- As árvores simétricas podem limitar a expressividade em relações muito complexas.
- Continua sendo um modelo sequencial e altamente "caixa-preta".

# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

## CATBOOST



# TÓPICO 4

## **STACKING E VOTING: COMBINANDO MODELOS DE DIFERENTES NATUREZAS**

# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

### STACKING

- Uma técnica de ensemble que combina as predições de múltiplos modelos **diferentes** através de um **meta-modelo**. O objetivo é aprender a melhor forma de combinar as predições, em vez de simplesmente usar a média ou a votação.
- **Funcionamento:**
  - **Divisão de Dados:** O conjunto de treino é dividido em K-folds (ex: 5 folds).
  - **Treinamento da Camada 0:**
    - Um conjunto de modelos base (ex: Random Forest, SVM, Regressão Logística) é treinado.
    - Para evitar vazamento de dados (data leakage), as predições para treinar o meta-modelo são geradas em dados que não foram vistos durante o treinamento (usando a validação cruzada dos K-folds).
  - **Criação do Novo Dataset:** Um novo dataset é criado onde as features são as predições dos modelos da Camada 0.

# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

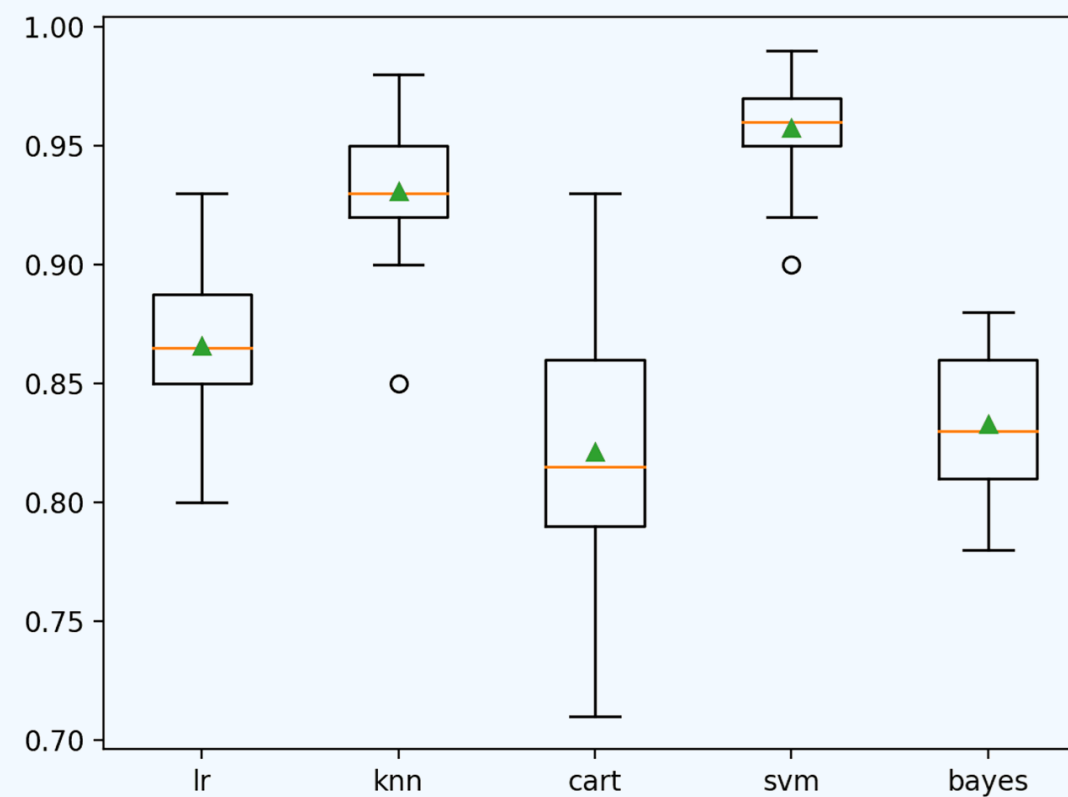
### STACKING

- **Treinamento da Camada 1 (Meta-Modelo):** Um meta-modelo (geralmente um modelo simples como Regressão Linear/Logística) é treinado neste novo *dataset*. Ele aprende os pesos ótimos para combinar as previsões da camada anterior.
- **Predição Final:** Para um novo dado, obtenha as previsões de todos os modelos da Camada 0 e alimente-as no meta-modelo para obter a predição final.
- **Pontos Positivos:**
  - Potencial para atingir o mais alto desempenho ao explorar as forças de tipos de modelos fundamentalmente diferentes.
  - Extremamente flexível.
- **Pontos Negativos:**
  - Muito complexo de implementar corretamente.
  - Custo computacional altíssimo.
  - Alto risco de *data leakage* se não for cuidadosamente implementado.
  - Interpretabilidade praticamente nula.

# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

## STACKING



```

1 # compare standalone models for binary classification
2 from numpy import mean
3 from numpy import std
4 from sklearn.datasets import make_classification
5 from sklearn.model_selection import cross_val_score
6 from sklearn.model_selection import RepeatedStratifiedKFold
7 from sklearn.linear_model import LogisticRegression
8 from sklearn.neighbors import KNeighborsClassifier
9 from sklearn.tree import DecisionTreeClassifier
10 from sklearn.svm import SVC
11 from sklearn.naive_bayes import GaussianNB
12 from matplotlib import pyplot
13
14 # get the dataset
15 def get_dataset():
16     X, y = make_classification(n_samples=1000, n_features=20, n_informative=15, n_redundant=5, random_state=1)
17     return X, y
18
19 # get a list of models to evaluate
20 def get_models():
21     models = dict()
22     models['lr'] = LogisticRegression()
23     models['knn'] = KNeighborsClassifier()
24     models['cart'] = DecisionTreeClassifier()
25     models['svm'] = SVC()
26     models['bayes'] = GaussianNB()
27     return models
28
29 # evaluate a given model using cross-validation
30 def evaluate_model(model, X, y):
31     cv = RepeatedStratifiedKFold(n_splits=10, n_repeats=3, random_state=1)
32     scores = cross_val_score(model, X, y, scoring='accuracy', cv=cv, n_jobs=-1, error_score='raise')
33     return scores
34
35 # define dataset
36 X, y = get_dataset()
37 # get the models to evaluate
38 models = get_models()
39 # evaluate the models and store results
40 results, names = list(), list()
41 for name, model in models.items():
42     scores = evaluate_model(model, X, y)
43     results.append(scores)
44     names.append(name)
45     print('> %s %.3f (%.3f)' % (name, mean(scores), std(scores)))
46 # plot model performance for comparison
47 pyplot.boxplot(results, labels=names, showmeans=True)
48 pyplot.show()

```

Trabalho

sklearn.ensemble.StackingClassifier

https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.StackingClassifier.html#sklearn.ensemble.StackingClassifier

scikit-learn

Install User Guide API Examples Community More ▾

Prev Up Next

scikit-learn 1.3.2  
Other versions

Please cite us if you use the software.

sklearn.ensemble.StackingClassifier  
StackingClassifier  
Examples using  
sklearn.ensemble.StackingClassifier

## sklearn.ensemble.StackingClassifier

```
class sklearn.ensemble.StackingClassifier(estimators, final_estimator=None, *, cv=None, stack_method='auto', n_jobs=None, passthrough=False, verbose=0)
```

Stack of estimators with a final classifier.

Stacked generalization consists in stacking the output of individual estimator and use a classifier to compute the final prediction. Stacking allows to use the strength of each individual estimator by using their output as input of a final estimator.

Note that `estimators_` are fitted on the full  $X$  while `final_estimator_` is trained using cross-validated predictions of the base estimators using `cross_val_predict`.

Read more in the [User Guide](#).

New in version 0.22.

**Parameters:**

**estimators : list of (str, estimator)**  
Base estimators which will be stacked together. Each element of the list is defined as a tuple of string (i.e. name) and an estimator instance. An estimator can be set to 'drop' using `set_params`.

The type of estimator is generally expected to be a classifier. However, one can pass a regressor for some use case (e.g. ordinal regression).

**final\_estimator : estimator, default=None**  
A classifier which will be used to combine the base estimators. The default classifier is a [LogisticRegression](#).

**cv : int, cross-validation generator, iterable, or "prefit", default=None**  
Determines the cross-validation splitting strategy used in `cross_val_predict` to train `final_estimator`. Possible inputs for cv are:

- None, to use the default 5-fold cross validation,
- integer, to specify the number of folds in a (Stratified) KFold,
- An object to be used as a cross-validation generator,
- An iterable yielding train, test splits,
- "prefit" to assume the `estimators` are prefit. In this case, the estimators will not be refitted.

For integer/None inputs, if the estimator is a classifier and  $y$  is either binary or multiclass, `StratifiedKFold` is used. In all other cases, `KFold` is used. These splitters are instantiated with `shuffle=False` so the splits will be the same across calls.

Refer [User Guide](#) for the various cross-validation strategies that can be used here.

If "prefit" is passed, it is assumed that all `estimators` have been fitted already. The `final_estimator_` is trained on the `estimators` predictions on the full training set and are **not** cross validated predictions. Please note that if the models have been trained on the same data to train the stacking model, there is a very high risk of overfitting.

New in version 1.1: The 'prefit' option was added in 1.1

**Note:** A larger number of split will provide no benefits if the number of training samples is large enough. Indeed, the training time will increase. cv is not used for model evaluation but for prediction.

**stack\_method : {'auto', 'predict\_proba', 'decision\_function', 'predict'}, default='auto'**  
Methods called for each base estimator. It can be:

sklearn.ensemble.StackingRegressor

Install User Guide API Examples Community More

Prev Up Next

skit-learn 1.3.2  
Other versions

Please cite us if you use the software.

sklearn.ensemble.StackingRegressor

StackingRegressor

Examples using

sklearn.ensemble.StackingRegressor

## sklearn.ensemble.StackingRegressor

```
class sklearn.ensemble.StackingRegressor(estimators, final_estimator=None, *, cv=None, n_jobs=None, passthrough=False, verbose=0)
```

Stack of estimators with a final regressor.

Stacked generalization consists in stacking the output of individual estimator and use a regressor to compute the final prediction. Stacking allows to use the strength of each individual estimator by using their output as input of a final estimator.

Note that `estimators_` are fitted on the full X while `final_estimator_` is trained using cross-validated predictions of the base estimators using `cross_val_predict`.

Read more in the [User Guide](#).

New in version 0.22.

**Parameters:**

- estimators : list of (str, estimator)**  
Base estimators which will be stacked together. Each element of the list is defined as a tuple of string (i.e. name) and an estimator instance. An estimator can be set to 'drop' using `set_params`.
- final\_estimator : estimator, default=None**  
A regressor which will be used to combine the base estimators. The default regressor is a `RidgeCV`.
- cv : int, cross-validation generator, iterable, or "prefit", default=None**  
Determines the cross-validation splitting strategy used in `cross_val_predict` to train `final_estimator`. Possible inputs for `cv` are:
  - None, to use the default 5-fold cross validation,
  - integer, to specify the number of folds in a (Stratified) KFold,
  - An object to be used as a cross-validation generator,
  - An iterable yielding train, test splits.
  - "prefit" to assume the `estimators` are prefit, and skip cross validation

For integer/None inputs, if the estimator is a classifier and `y` is either binary or multiclass, `StratifiedKFold` is used. In all other cases, `KFold` is used. These splitters are instantiated with `shuffle=False` so the splits will be the same across calls.

Refer [User Guide](#) for the various cross-validation strategies that can be used here.

If "prefit" is passed, it is assumed that all `estimators` have been fitted already. The `final_estimator_` is trained on the `estimators` predictions on the full training set and are **not** cross validated predictions. Please note that if the models have been trained on the same data to train the stacking model, there is a very high risk of overfitting.

New in version 1.1: The "prefit" option was added in 1.1

**Note:** A larger number of split will provide no benefits if the number of training samples is large enough. Indeed, the training time will increase. `cv` is not used for model evaluation but for prediction.

**n\_jobs : int, default=None**  
The number of jobs to run in parallel for fit of all estimators. None means 1 unless in a `joblib.parallel_backend` context. -1 means using all processors. See Glossary for more details.

**passthrough : bool, default=False**

# PRINCIPAIS ALGORITMOS

## – ENSEMBLE

### RESUMO

| CARACTERÍSTICA         | BAGGING                                       | RANDOM FOREST                               | ADABOOST                                         | GBM / XGBOOST / LIGHTGBM                             | STACKING                                        | CATBOOST                                              |
|------------------------|-----------------------------------------------|---------------------------------------------|--------------------------------------------------|------------------------------------------------------|-------------------------------------------------|-------------------------------------------------------|
| Ideia Principal        | Média de modelos em amostras <i>bootstrap</i> | Bagging de árvores + amostragem de features | Sequencial, foca nos erros via peso das amostras | Sequencial, foca nos erros via resíduos (gradientes) | Meta-modelo aprende a combinar modelos diversos | Sequencial com Ordered Boosting e categóricas nativas |
| Impacto no Viés        | Não altera                                    | Não altera                                  | Reduz                                            | Reduz Fortemente                                     | Reduz                                           | Reduz Fortemente                                      |
| Impacto na Variância   | Reduz                                         | Reduz Fortemente                            | Pode aumentar                                    | Controla com regularização                           | Reduz                                           | Controla (regularização)                              |
| Custo Computacional    | Médio (Paralelizável)                         | Médio-Alto (Paralelizável)                  | Médio (Sequencial)                               | Alto (Sequencial com otimizações)                    | Muito Alto                                      | Alto (predição rápida)                                |
| Custo de Implementação | Baixo                                         | Baixo                                       | Médio                                            | Alto (muitos parâmetros)                             | Muito Alto                                      | Baixo (bons defaults)                                 |
| Interpretabilidade     | Baixa                                         | Baixa (mas tem <i>feature importance</i> )  | Baixa                                            | Muito Baixa                                          | Nula                                            | Muito Baixa                                           |
| Principal Vantagem     | Simplicidade, reduz <i>overfitting</i>        | Ótimo desempenho " <i>out-of-the-box</i> "  | Eficaz com <i>weak learners</i> muito simples    | Performance de ponta, flexibilidade                  | Potencialmente a maior acurácia                 | Forte com categóricas, pouco ajuste                   |
| Principal Desvantagem  | Não melhora o viés                            | "Caixa-preta"                               | Sensível a ruído e outliers                      | Complexidade de ajuste                               | Complexidade de implementação, custo            | Treino lento, usa muita memória                       |